



内容 Contents

■ Section 1	软件项目管理概述	2
■ Section 2	项目初始	3
■ Section 3	软件项目范围计划	4
■ Section 4	软件项目成本计划	2
■ Section 5	软件项目进度计划	3
■ Section 6	软件项目质量计划	3
■ Section 7	软件配置管理计划	2
■ Section 8	软件项目团队计划	2
■ Section 9	软件项目风险计划	2
■ Section 10	软件项目合同计划	2
■ Section 11	项目执行控制	6
■ Section 12	项目结束	2



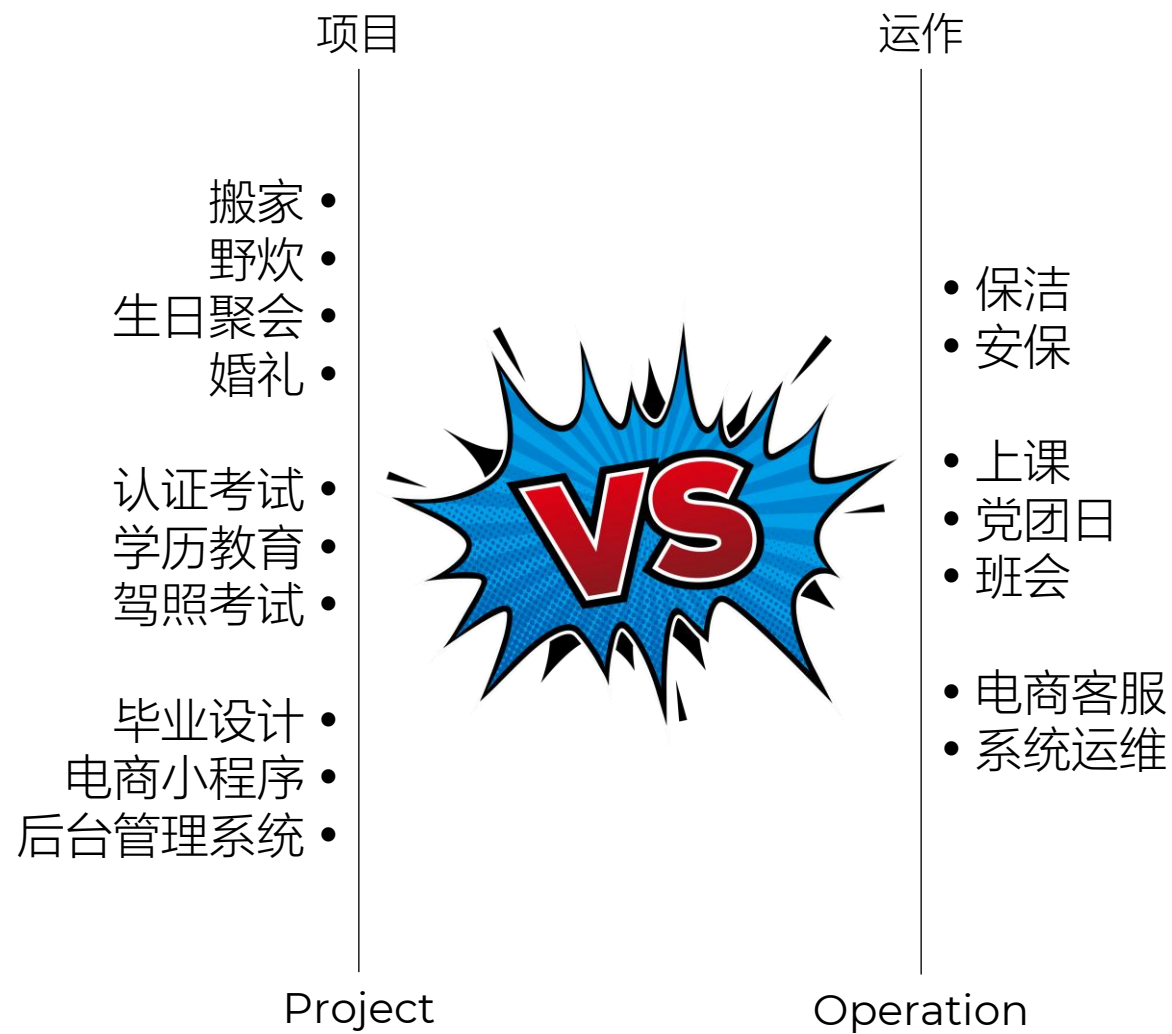


1. 项目与软件项目
2. 项目管理与软件项目管理
3. 传统软件项目管理
4. 敏捷软件项目管理

软件项目管理概述

Section 1 Overview

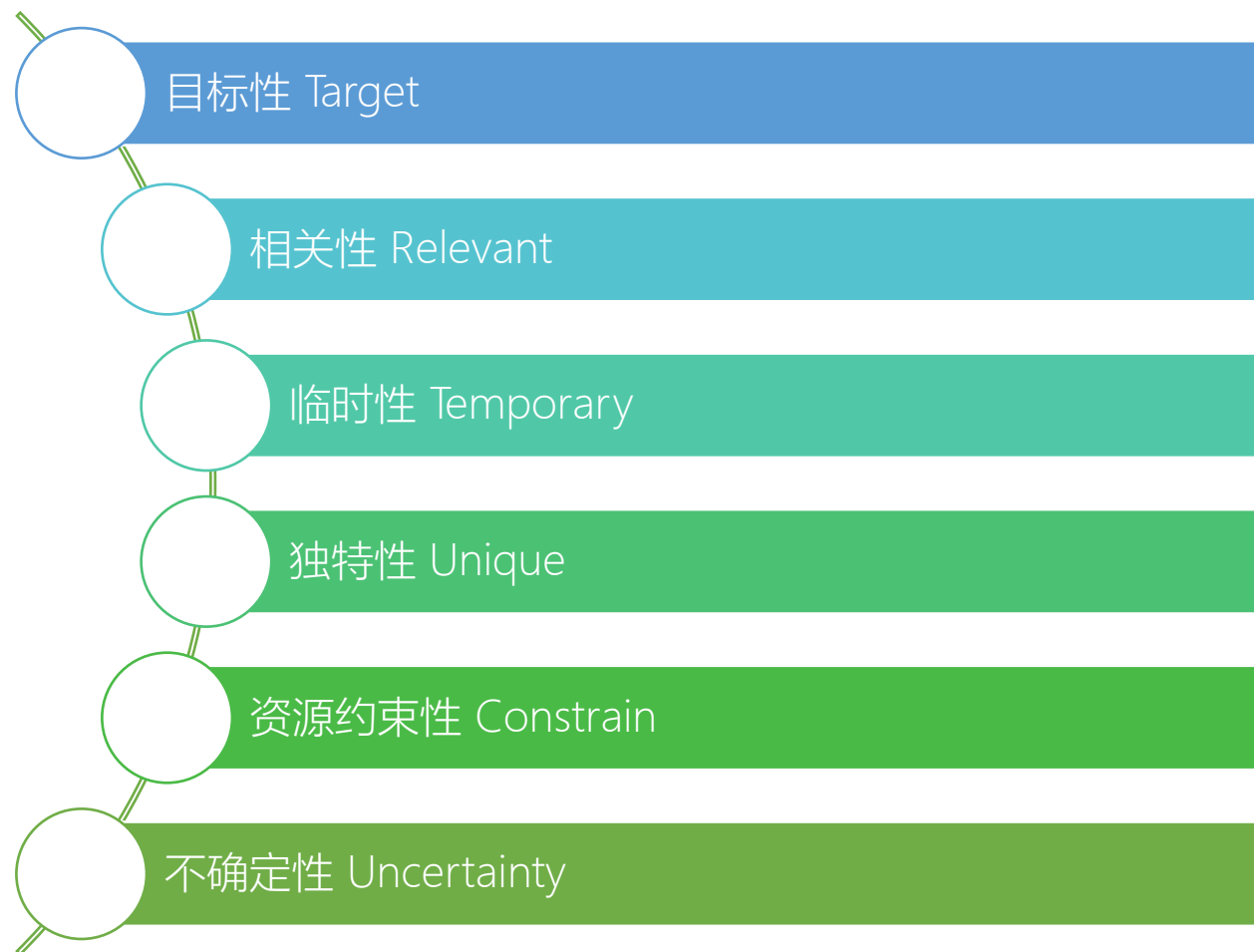
- 定义 Concept:
 - 为了创建一个唯一的产品或提供一个唯一的服务而进行的临时性的努力
 - A project is a temporary endeavor undertaken to create a unique product, service, or **result**.



1.1 项目

Project

- 特征 Features



1.2 子项目

Sub-Program

- 将项目分解为更小的单位
- 便于更好的控制项目
- 划分很灵活



1.3 项目集

Program

- 一组相互关联的项目、子项目和受控过程的活动
- 也称为大型项目
- 目的：通过协同管理获取单独管理各个部分时无法实现的额外效益
- 相互关联的项目：多个项目之间存在逻辑或目标上的联系
- 协调管理：统一规划、组织与控制，以确保整体一致性
- 核心价值：整体大于部分之和



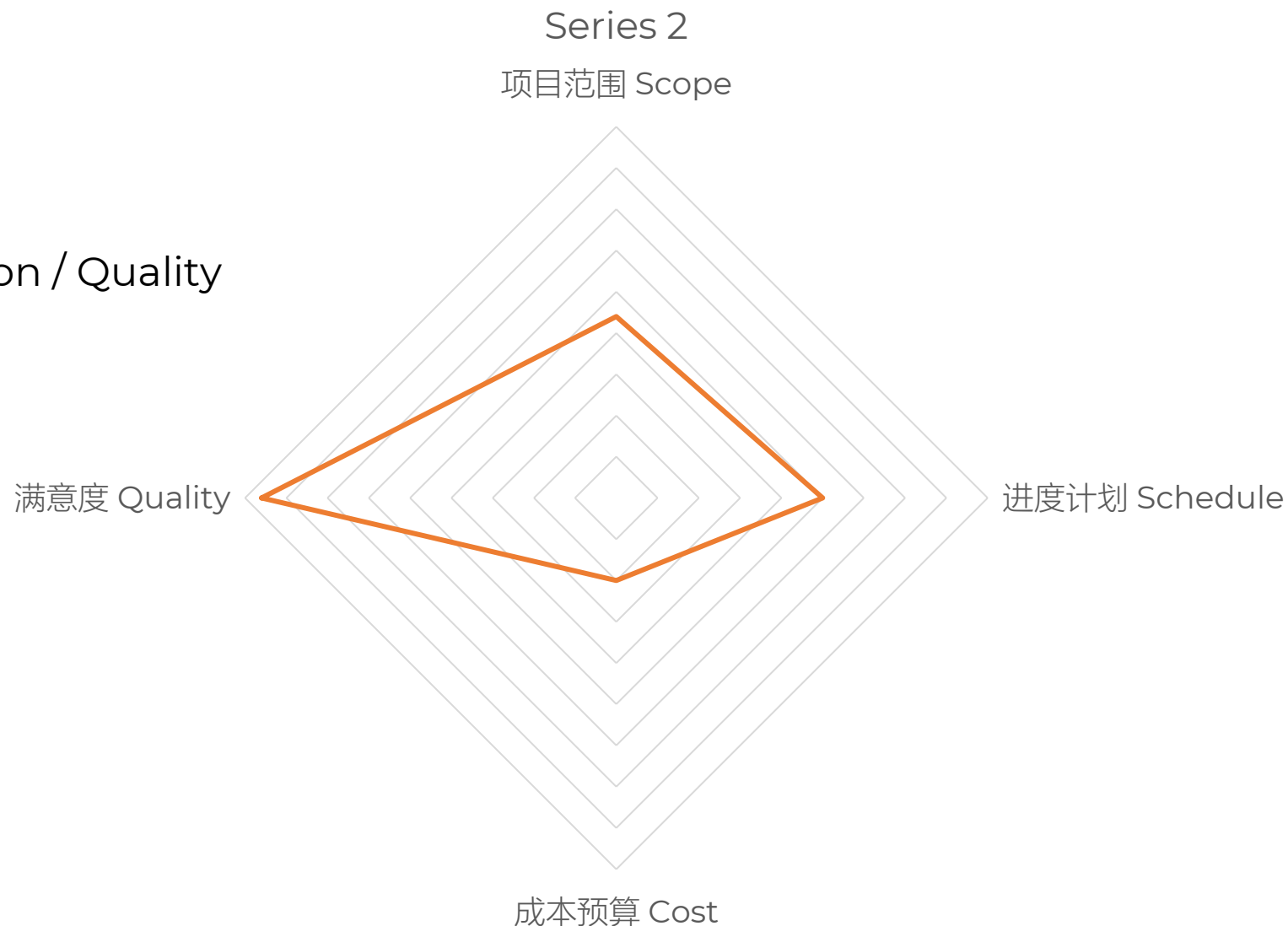
- 项目组合是为实现战略目标而整合管理的一系列项目、项目集和运营活动的集合
- 特点
 - 项目或项目集之间不一定彼此依赖或直接相关
 - 管理重点在于战略对齐，而非技术或执行上的关联性
- 战略导向：项目组合的管理是为了达成组织的战略目标
- 多样化组成：包含项目、项目集、子项目组合及运营工作
- 非依赖性：组合内的元素可以独立运作，无需相互依赖
- 统一管理：通过集中管理实现资源优化、优先级排序和战略一致性
- 强调
 - 战略价值和整体效益



- 含义 Concept
 - 计算机程序以及说明程序的各种文档
- 组成要素 Component
 - 程序
 - 数据
 - 相关文档
- 特点 Feature
 - 逻辑实体
 - 同硬件相比，没有生产、磨损和老化
 - 开发受计算机系统限制
 - 技术复杂、更新迭代快
 - 开发成本低和学习成本高
 - 涉及多方面因素
 - 在AI盛行的当下，仍然严重依赖人工

1.5 软件项目

- 制约 Constrain
 - 项目范围 Scope
 - 进度计划 Schedule / Time
 - 成本 Cost / Budget
 - 用户满意度 User Satisfaction / Quality



1.5 软件项目

- 趋势 Trend
 - 当前软件项目开发正从“本地部署、一次性交付”转向“云端托管、持续迭代、服务化运营”
 - SaaS + Cloud 已成主流范式，推动软件产业向更灵活、高效、智能的方向演进
 - 基于云平台构建 SaaS 应用，实现“开箱即用、按需付费、全球可达”

Item	English	Chinese	Description	Relevant
IaaS	Infrastructure as a Service	基础设施即服务	提供虚拟机、存储、网络等底层资源（如 AWS EC2、阿里云 ECS）	SaaS 可构建在 IaaS 之上
PaaS	Platform as a Service	平台即服务	提供开发、部署、运行环境（如 Google App Engine、Heroku）	SaaS 开发者常使用 PaaS 加速开发
SaaS	Software as a Service	软件即服务	直接面向终端用户的完整应用（如钉钉、Salesforce、Notion）	最终交付形态



- 例子 Example
 - 本地 IDE vs 云开发; [CodePen](#)
 - DIY 自建服务器 vs 云服务器; 阿里云 ECS, 腾讯云 CVM, 亚马逊 AWS
 - 传统办公 vs OA; 腾讯文档、金山文档、钉钉、飞书
 - 其它: [Supabase](#)、[GitHub](#)

没有必要为了喝口奶而养一头牛

Why buy the cow when you only need the milk?

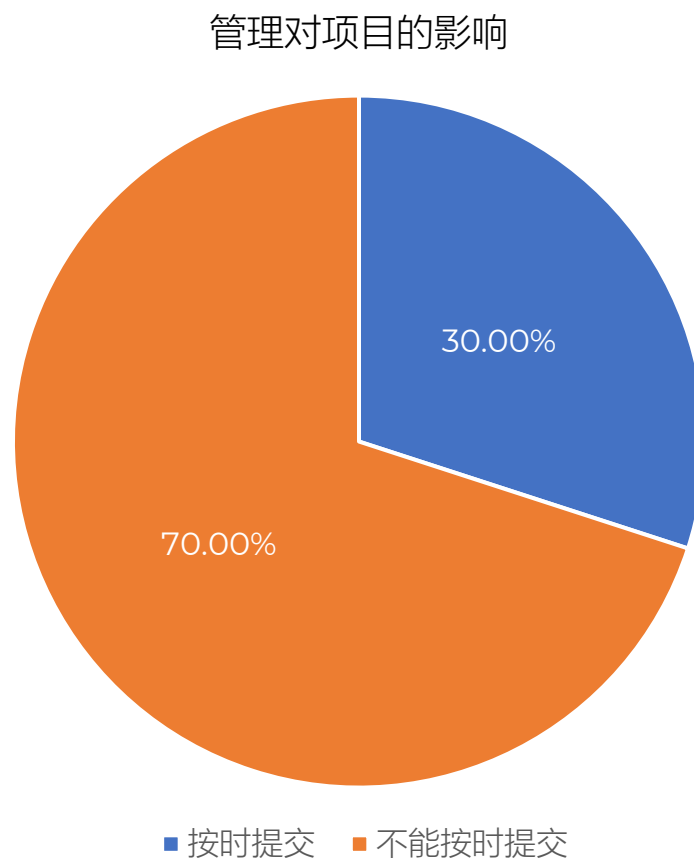


- 项目管理是一定的主体，为了实现目标，利用各种有效手段，对执行中的项目周期的各个阶段进行计划、组织、协调、指挥、控制，以取得良好经济效益的各项活动的总和



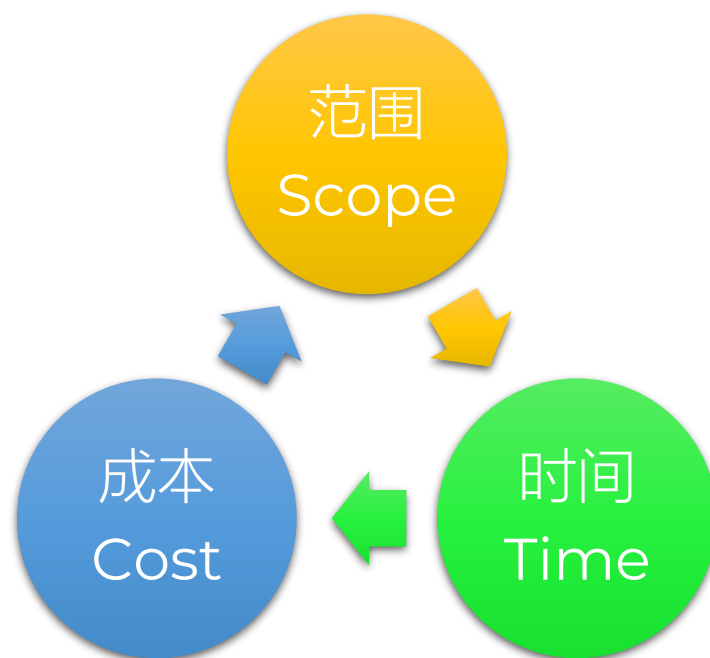
2.2 软件项目管理

- 软件工程组成部分
- 确保软件项目满足预算，成本等约束，提交高质量软件产品



2.2 软件项目管理

- 项目管理铁三角



3 传统软件项目管理

PMBOK

- 基于经验，现在需要系统学习
- PMP
 - 100多个国家认可
 - 基于 PMBOK 7th，同时结合了《敏捷实践指南》。
 - 考试中很少再考某个过程的输入输出是什么（ITTO记忆题大幅减少）
 - 更多考的是情景题：面对这种情况，项目经理应该秉持什么原则？优先关注哪个绩效域？
 - 敏捷和混合题型占比高达 50%

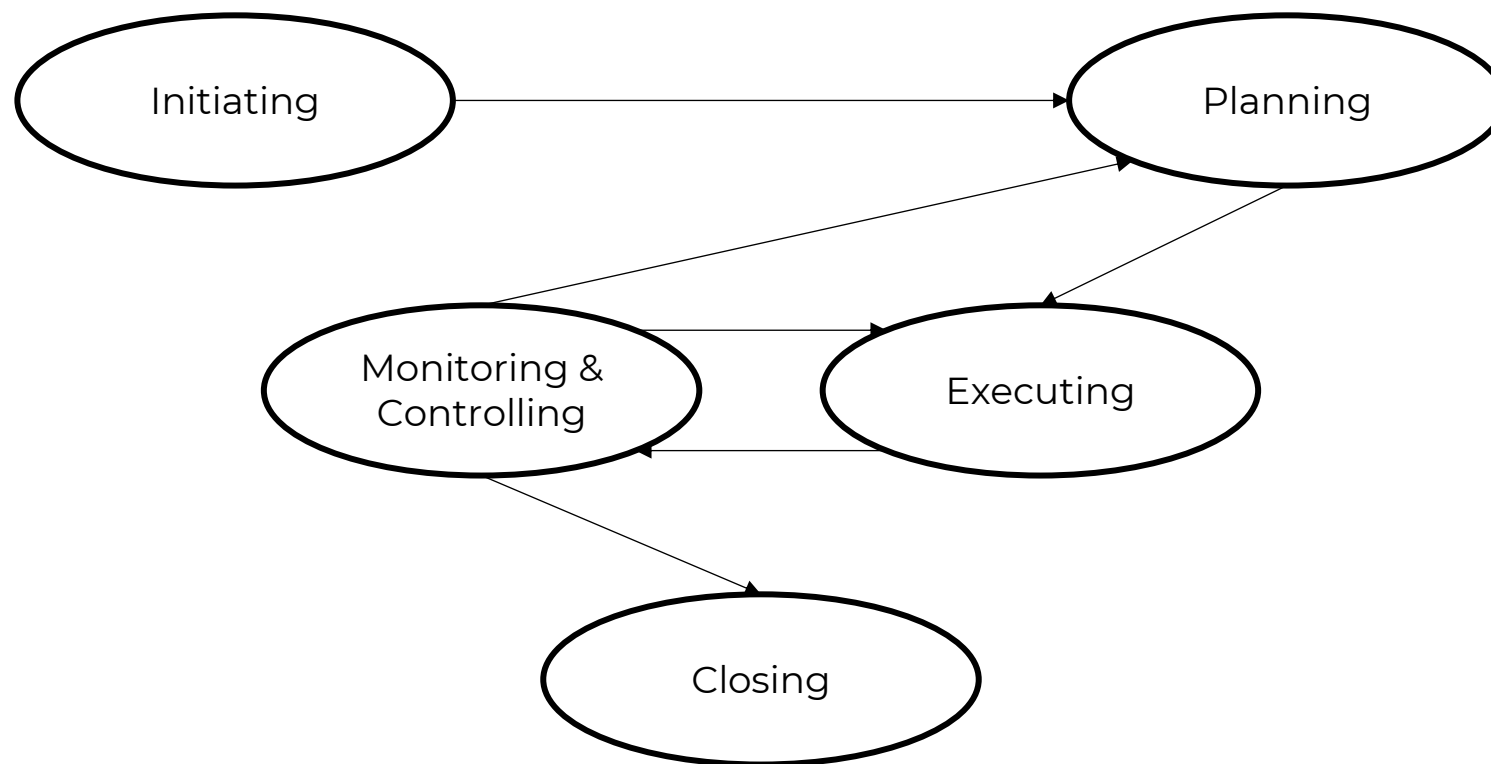
Item	Title	Description
PMP	Project Management Professional	项目管理专业资质认证
PMI	Project Management Institute	项目管理委员会
PMBOK 6 th	Project Management Body of Knowledge, 6th Edition	项目管理知识体系，第6版
PMBOK 7 th	Project Management Body of Knowledge, 7th Edition	项目管理知识体系，第7版



Aspect	PMBOK 6th	PMBOK 7th
Philosophy	基于过程 Process-Based	基于原则 Principle-Based
Structure	5大过程组 + 10大知识领域	12项原则 + 8大绩效域
Focus	产出 Outputs 交付了什么文件/产品?	成果 Outcomes & 价值 Value 实现了什么业务目标?
Methodology Scope	偏向预测型 (瀑布式)	全生命周期适用 (预测型、敏捷型、混合型融合)
PM role	执行者、控制者、流程管理者	服务型领导、变革推动者、价值交付者
Tailoring	较僵化, 强调遵循标准流程	高度灵活, 强调根据环境裁剪 (Tailoring)
Definition of Success	按时、按预算、按范围交付 (铁三角)	交付价值、满足干系人期望、实现战略目标
Format & Length	厚书, 充满ITTO (输入/工具/输出) 表格	更薄, 更像指导手册, 去除了大量ITTO细节
In a word	Do the project right	Do the right project



- 2017年9月发布
- 传统项目管理方法的集大成者，强调怎么做 How to do
- 基于过程 Process-Based
- 核心理念 Philosophy
 - 合规性与标准化：强调遵循既定的流程和规范
 - 可交付成果导向：关注是否按时、按预算完成了规定的产出
 - 预测型为主：虽然包含了敏捷内容（附录），但主体逻辑仍偏向瀑布式（预测型）管理
- 核心架构 Structure
 - 五大标准过程组 (5 Process Groups)：按时间顺序排列的项目生命周期阶段
 - 十大知识领域 (10 Knowledge Areas)：按专业职能划分的管理领域
- 缺点 Drawback
 - 为了走流程而走流程，忽视实际效果



3.1.1 五大标准过程组

5 Process Groups/PMBOK 6th

- 含49个过程（表1-1， P19）

Process Group	Description	Process
1 启动 Initiating	定义新项目或阶段，获得授权 关键输出：项目章程、识别干系人	2
2 规划 Planning	制定范围、进度、成本、质量、风险等计划 过程占比最大，强调滚动式规划	24
3 执行 Executing	完成项目工作，满足范围说明书要求 包括团队建设、质量保证、采购实施等	10
4 监控 Monitoring & Controlling	跟踪、审查和调整项目进展与绩效 核心：变更控制和偏差分析	12
5 结束 Closing	正式结束项目或阶段 移交成果，总结经验	1



3.1.2 十大知识领域

10 Knowledge Areas/PMBOK 6th

- 每个领域涉及若干过程
- 每个过程都有详细的 ITTO (输入 Input、工具 Tools 与技术 Techniques、输出 Output)
 - 如成本管理 (图1-8, P14)

Areas		Description
1	项目集成管理 Integration	统一协调各过程, 如制定章程、项目管理计划、变更控制
2	项目范围管理 Scope	定义和控制做哪些工作, 防止范围蔓延
3	项目进度管理 Schedule	活动定义、排序、估算、制定进度计划
4	项目成本管理 Cost	估算、预算、控制成本
5	项目质量管理 Quality	规划质量、管理质量、控制质量
6	项目资源管理 Resource	获取、发展、管理团队及实物资源
7	项目沟通管理 Communication	规划、管理、监督沟通
8	项目风险管理 Risk	识别、分析、应对不确定性
9	项目采购管理 Procurement	从外部获取产品/服务
10	项目干系人管理 Stakeholder	识别干系人、制定参与策略、管理期望



- 2021年7月发布
- 强调做什么 What 和为什么做 Why
- 基于原则 Principle-Based
- 核心理念 Philosophy
 - 价值导向：不仅关注产出 Output，更关注成果 Outcome 和商业价值
 - 灵活性与裁剪：没有一刀切的流程，鼓励根据项目情境选择合适的方法（敏捷、混合或预测）
 - 系统性思维：将项目视为复杂系统的一部分，强调应对不确定性和变革
- 核心架构 Structure
 - 十二项原则 (12 Principles)：指导项目行为的核心价值观，适用于任何方法论（预测、敏捷、混合）
 - 八大绩效域 (8 Performance Domains)：取代了知识领域，强调这些领域需要同时关注、相互作用，以交付项目成果

- 注重价值交付和适应性
- 不是对第6版推翻、不是替代第6版
- 而是对第6版中项目管理核心精髓的延伸，是对第6版知识领域与过程组高层次、高维度的承接，再度续写下一个项目管理标准构架的辉煌
- 与第6版长期共存
- PMBOK 第八版（2025年11月发布）把8大绩效域又改成了7大绩效域
- 体系框架动态发展

3.2.1 十二项原则

12 Principles/PMBOK 7th

1	尽职、尊重和关心(管家精神)	Be a diligent, respectful, and caring steward .	核心：建立信任与 责任感 。 项目经理应像“管家”一样，以诚信、关怀和尊重的态度管理资源（包括人、财、物和环境），严守合规与安全底线，追求长期可持续性。
2	营造协作的项目团队环境	Create a collaborative project team environment.	核心：团队效能高于个人总和。 创造一个心理安全、开放沟通的环境，鼓励团队成员共享知识、互相支持。高效的协作能激发创新，提升解决问题的能力。
3	有效参与相关方	Effectively engage with stakeholders.	核心：双向沟通与期望管理。 主动 识别相关方，理解他们的需求和期望，并通过持续的对话建立伙伴关系。成功的项目取决于相关方的支持和参与，而不仅仅是交付产品。
4	聚焦于价值	Focus on value.	核心：价值是最终目标。 项目的存在是为了创造价值（商业价值、社会效益等）。所有决策和行动都应以“是否最大化价值”为衡量标准， 避免为了做项目而做项目 。
5	识别、评估和响应系统交互	Recognize, evaluate, and respond to system interactions.	核心：系统思维 (Systems Thinking)。 项目不是孤立存在的，它处于更大的组织和社会系统中。必须理解项目内部各要素之间，以及项目与外部环境之间的复杂互动关系，以预测连锁反应。
6	展现领导力行为	Demonstrate leadership behaviors.	核心：领导力人人可为。 领导力不仅仅是项目经理的职权，而是一种行为。无论角色如何，每个人都应在适当的时候展现领导力，激励团队、解决冲突并指引方向。



3.2.1 十二项原则

12 Principles/PMBOK 7th

7	根据情境进行裁剪	Tailor based on context.	核心：没有“一刀切”的方法。 每个项目都是独特的。必须根据项目的规模、复杂性、不确定性、相关方需求及组织文化，灵活调整方法、流程和工件（无论是敏捷、预测还是混合模式）。
8	将质量融入过程和可交付成果	Build quality into processes and deliverables.	核心：预防胜于检查。 质量不是最后检验出来的，而是规划和执行出来的。通过优化流程和标准，确保可交付成果从一开始就符合要求和预期，减少返工。
9	驾驭复杂性	Navigate complexity.	核心：在混乱中寻找路径。 项目常面临人类行为、系统技术或环境模糊性带来的复杂性。管理者需利用经验、直觉和分析工具，在不确定中做出决策，化繁为简。
10	优化风险应对	Optimize risk responses.	核心：拥抱不确定性。 风险既包含威胁也包含机会。不仅要规避负面风险，更要主动捕捉正面风险。通过持续的风险管理，将不确定性对项目目标的影响降至最低或转为优势。
11	拥抱适应性和韧性	Embrace adaptability and resiliency.	核心：在变化中生存与发展。 面对快速变化的环境，团队需要具备适应性（灵活调整方法）和韧性（从挫折中恢复的能力）。要求建立反馈循环，快速学习并迭代。
12	为实现目标而驱动变革	Enable change to achieve the envisioned future state.	核心：项目即变革。 项目的目的是推动组织从“当前状态”走向“未来愿景”。不仅要交付产出，还要管理变革过程，帮助人们接受新方式，确保成果被真正采用并产生效益。



3.2.2 八大绩效域

8 Performance Domains/PMBOK 7th

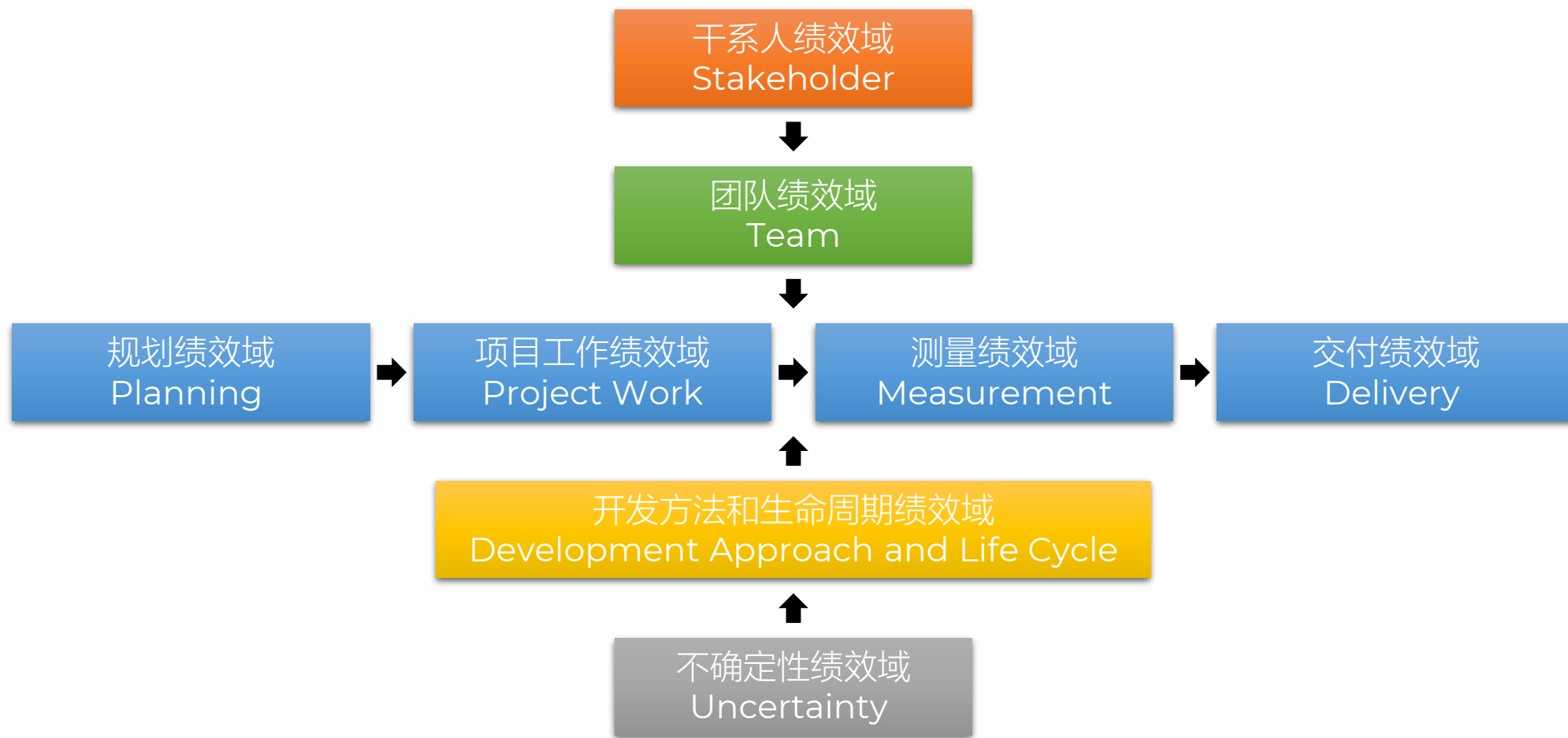
- 项目在实现预期成果和价值方面的整体表现 **Performance**
- 一组相互关联的项目活动，对于有效交付项目成果至关重要
- 同时发生、动态交织、需要持续关注

	English	Chinese	Description
1	Stakeholder Performance Domain	相关方绩效域	管理与相关方的互动，建立共识，确保持续参与和支持。
2	Team Performance Domain	团队绩效域	发展团队技能，促进协作，解决冲突，提升团队绩效。
3	Development Approach and Life Cycle Performance Domain	开发方法和生命周期绩效域	选择适合项目的开发方法（预测、迭代、增量、敏捷或混合）及生命周期阶段。
4	Planning Performance Domain	规划绩效域	制定初始及持续的计划，协调范围、进度、成本等，以指导项目执行。
5	Project Work Performance Domain	项目工作绩效域	管理实际的项目执行工作，包括资源分配、沟通、问题解决和知识管理。
6	Delivery Performance Domain	交付绩效域	确保项目产出符合预期，满足质量要求，并最终实现预期的商业价值。
7	Measurement Performance Domain	测量绩效域	评估项目绩效，通过指标和数据了解项目健康状况，支持决策。
8	Uncertainty Performance Domain	不确定性绩效域	应对风险、模糊性、复杂性和易变性（VUCA），制定应对策略。不是消除（因为不可能）而是驾驭，将其转化为机会或可控的损失。



3.2.2 八大绩效域

8 Performance Domains/PMBOK 7th



VUCA

- 描述当今商业环境的特征
- PMBOK 指南的演变史，就是一部对抗 VUCA 的历史

Char	English	Chinese	Description
V	Volatility	易变性	变化发生的速度极快，幅度极大 计划赶不上变化，需求频繁变更
U	Uncertainty	不确定性	缺乏信息，难以预测未来，因果关系不明 风险难以识别，传统预测失效
C	Complexity	复杂性	影响因素众多，相互交织，牵一发而动全身 单一解决方案无效，需要系统思考
A	Ambiguity	模糊性	情况模棱两可，容易产生误解，没有先例可循 目标不清晰，需求难以定义



一家电商公司要在“双11”前上线一个新的促销系统。距离上线只剩3周，突然核心开发人员离职，且发现了一个严重的安全漏洞。
以下哪些做法体现了PMBOK7的思想？

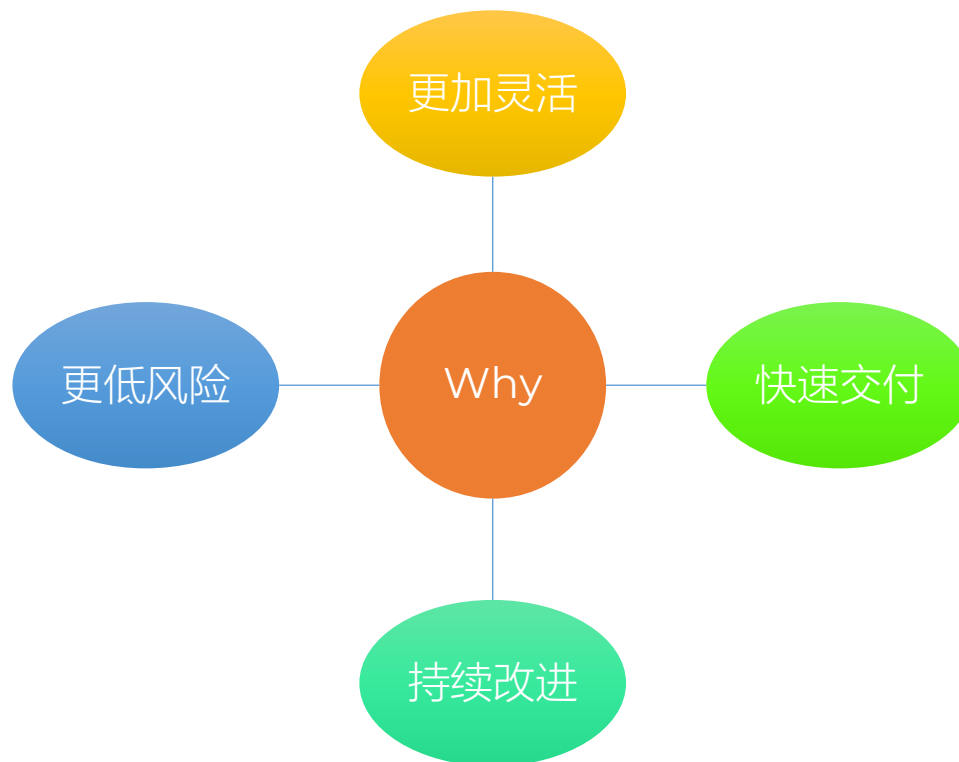
- A：打开《风险登记册》，走变更流程，申请延期
- B：按“资源管理计划”去招聘新人
- C：立即承认原计划已失效。不再试图消除风险，而是启动应急预案
- D：立刻找CEO和业务方沟通，获得授权，调整范围
- E：重新定义最小可行产品 MVP，保证双11当天能下单、能支付
- F：召开紧急动员会，透明化危机；邀请资深架构师临时顶岗

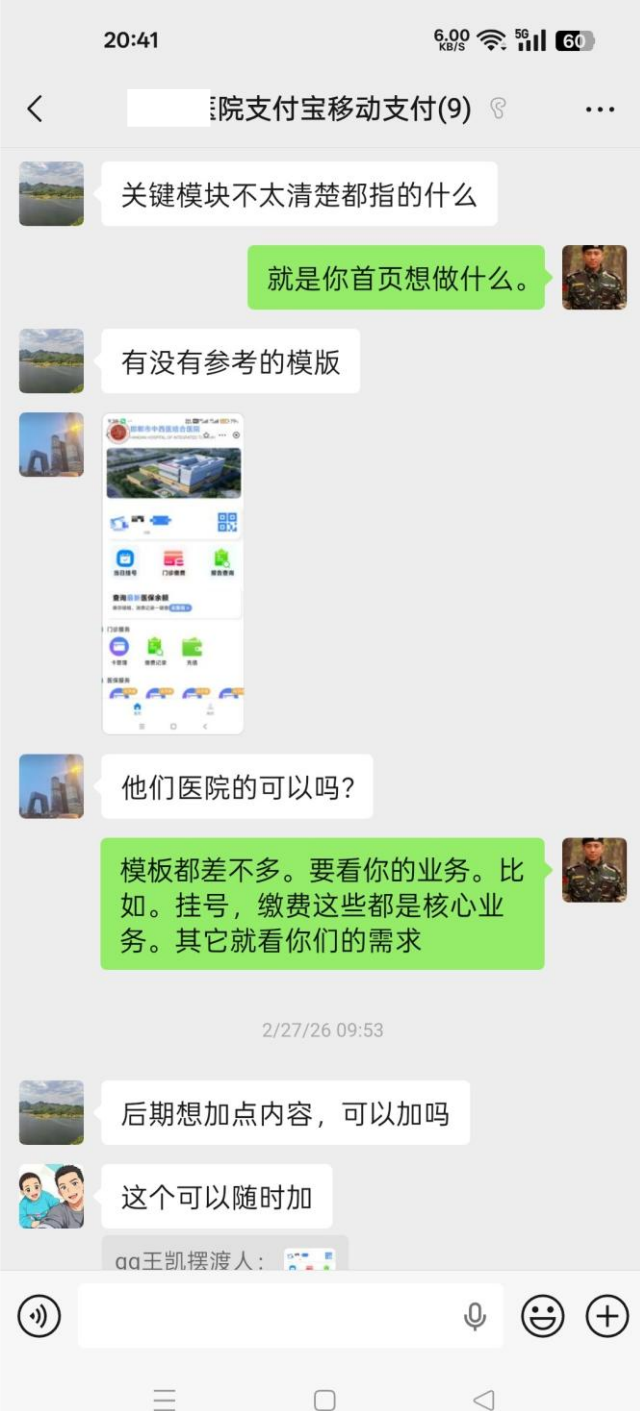
例题
Sample



4.1 敏捷开发

- 软件项目快速多变的环境要求快速的开发和提交
 - 部分可用、阶段可用、多次交付
 - 试水
 - 看市场反映
 - 及时止损

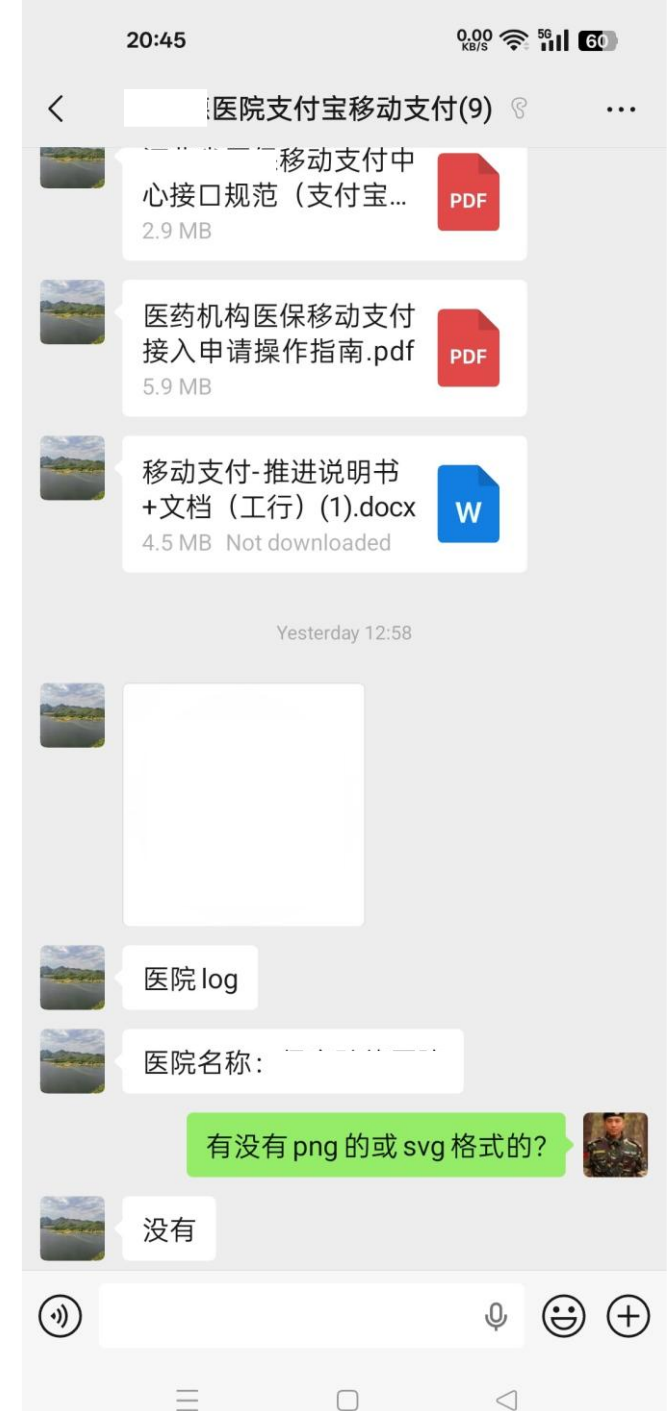


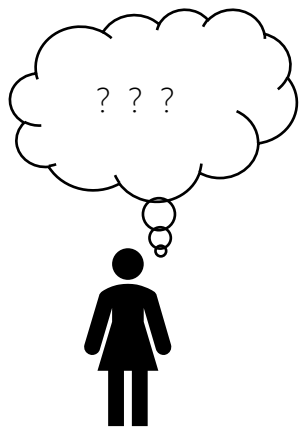


Software Project

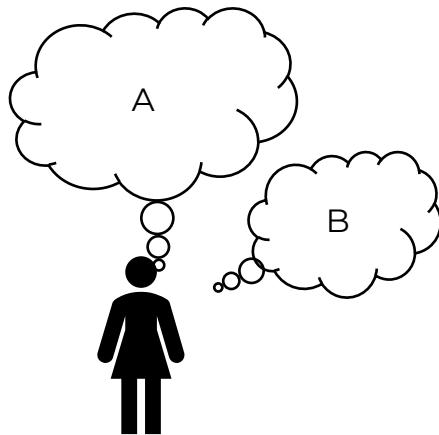


ub.io

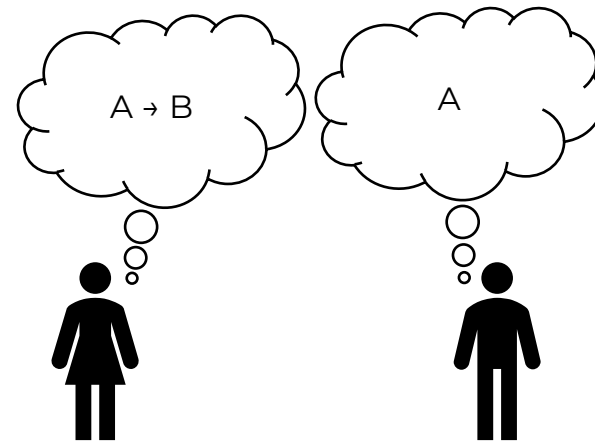




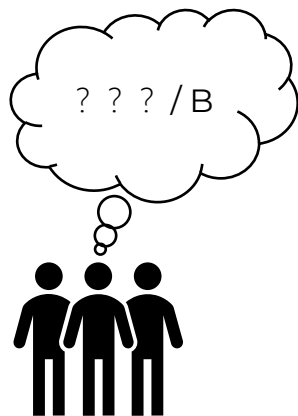
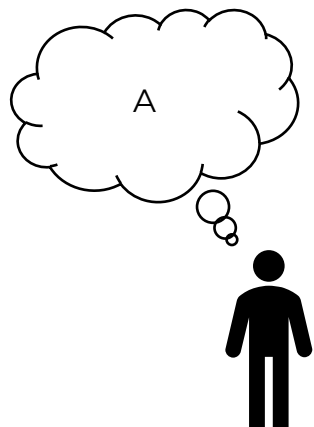
Case 1 Client 不知道需求



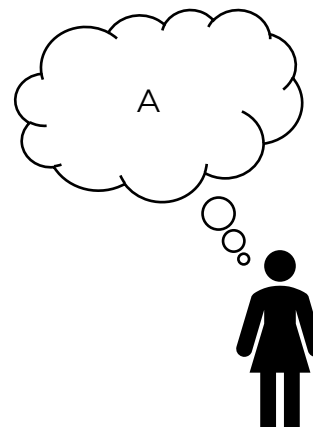
Case 2 Client 知道需求但是表达错误



Case 3 Client 知道并清晰传达需求
但是交互时，需求发生变化



Case 4 团队无法领悟或理解有偏差



Case 5 大环境变化、发生不可抗力



4.2 敏捷软件开发宣言

- Manifesto for Agile **Software** Development
- 简称敏捷宣言
- 聚焦软件项目管理
- 2001年，17位软件开发专家在美国犹他州雪鸟滑雪度假村聚会，共同起草
- 应对迅速变化需求的快速软件开发方法
- 是一种迭代、循序渐进的开发方法
- 体现的是一种思维模式 Thinking
- <https://agilemanifesto.org/>

敏捷是创造并响应变化，从而在动荡的商业环境中创造利润的能力。敏捷是平衡灵活性和稳定性的能力。

——吉姆·海斯密斯，2002



4.2.1 敏捷宣言四个核心价值

4 Values/Agile Manifesto



Values		Description
1	Individuals and interactions over processes and tools	以人为本
2	Working software over comprehensive documentation	以价值为导向
3	Customer collaboration over contract negotiation	合作共赢
4	Responding to change over following a plan	拥抱变化



4.2.2 敏捷宣言十二项原则

12 Principles/Agile Manifesto

- 第 1-3 条：关注“做什么”（交付价值、应对变化、快速产出）
- 第 4-6 条：关注“谁来做”和“怎么做”（人、沟通、信任）

	English	Chinese	Key Focus
1	Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.	我们的最高目标，是通过尽早并持续交付有价值的软件来满足客户。	价值交付 Value & Delivery
2	Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.	欢迎需求变化，即使是在开发后期。敏捷过程善于利用变化来为客户创造竞争优势。	拥抱变化 Embrace Change
3	Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.	频繁地交付可工作的软件，周期可以从几周到几个月，倾向于更短的周期。	频繁迭代 Frequency
4	Business people and developers must work together daily throughout the project.	业务人员和开发人员必须在整个项目期间每天一起工作。	协作沟通 Collaboration
5	Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.	围绕有积极性的个体构建项目。给予他们所需的环境和支持，并信任他们能够完成工作。	信任与激励 Trust & Motivation
6	The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.	面对面交谈是团队内部传递信息的最有效方式。	高效沟通 Face-to-Face



4.2.2 敏捷宣言十二项原则

- 第 7-9 条：关注“做得怎么样”（进度标准、节奏、质量）
- 第 10-12 条：关注“如何进化”（做减法、自组织、复盘）

	English	Chinese	Key Focus
7	Working software is the primary measure of progress.	可工作的软件是衡量进展的首要标准。	结果导向 Working Software
8	Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.	敏捷过程提倡可持续的开发节奏。赞助者、开发者和用户应该能够长期保持恒定的速度。	可持续发展 Sustainability
9	Continuous attention to technical excellence and good design enhances agility.	持续关注技术卓越和良好设计，以增强敏捷能力。	技术卓越 Technical Excellence
10	Simplicity —the art of maximizing the amount of work not done—is essential.	简洁 - 最大化未完成工作量的艺术 - 是根本的。	极简主义 Simplicity
11	The best architectures, requirements, and designs emerge from self-organizing teams.	最好的架构、需求和设计出自自组织团队。	自组织 Self-Organization
12	At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behavior accordingly.	团队定期反思如何变得更高效，并相应地调整自身行为。	持续改进 Reflection & Adjustment



维度	敏捷 12 原则	PMBOK 7 的 12 原则
适用范围	狭义 主要针对软件开发、IT 项目	广义 适用于建筑、医疗、活动、软件、制造等所有行业
关注点	怎么做软件 强调迭代、代码、客户协作	怎么管项目 强调价值、系统、道德、领导力、不确定性
语言风格	行动导向 交付软件、欢迎变更、每天一起	思维导向 聚焦价值、系统思考、驾驭复杂性
对瀑布的态度	隐含对立 旨在解决瀑布的痛点	包容融合 认为瀑布、敏捷、混合都是工具，视情况裁



课后思考 Homework

是不是所有的项目都应该采取敏捷开发？





1. 项目评估
2. 项目立项
3. 项目招投标
4. 项目章程
5. 软件项目生存期模型
6. Scrum

项目初始

Section 2 Project Initialization



- 项目启动
- 项目可行性分析
- 项目经济性分析
- 目的
 - 初步判断项目的可行性、必要性、资源需求等。
 - 属于前期调研/预研性质
- 输出
 - 项目评估报告

2.1.1 净现金值

NPV

- Input_t : 第t年的现金流入量
- Output_t : 第t年的现金流出量
- CF_t : Current Flow at time t, 当前净现金流 = $I_t - O_t$
- r : 贴现率
- NPV: Net Present Value, 净现金值

$$NPV = \sum_{t=0}^n \frac{\text{CF}_t}{(1+r)^t}$$



2.1.1 净现金值

NPV

- 例1. 某软件公司准备投标某项目，估计成本是75.95万，项目的合同金额100万，分2次支付，项目开始支付50万，项目结束50万，项目周期2年，贴现率为8%，请计算NPV。

t	Input流入	Output流出	Input-Output净值	NPVt
t0	50	75.95	-25.95	-25.95
t1	0	0	0	0
t2	50	0	50	$50/(1+8\%)^2$

$$NPV = \sum_{t=0}^n \frac{CF_t}{(1+r)^t} = -25.95 + 0 + \frac{50}{(1+8\%)^2}$$



2.1.1 净现金值

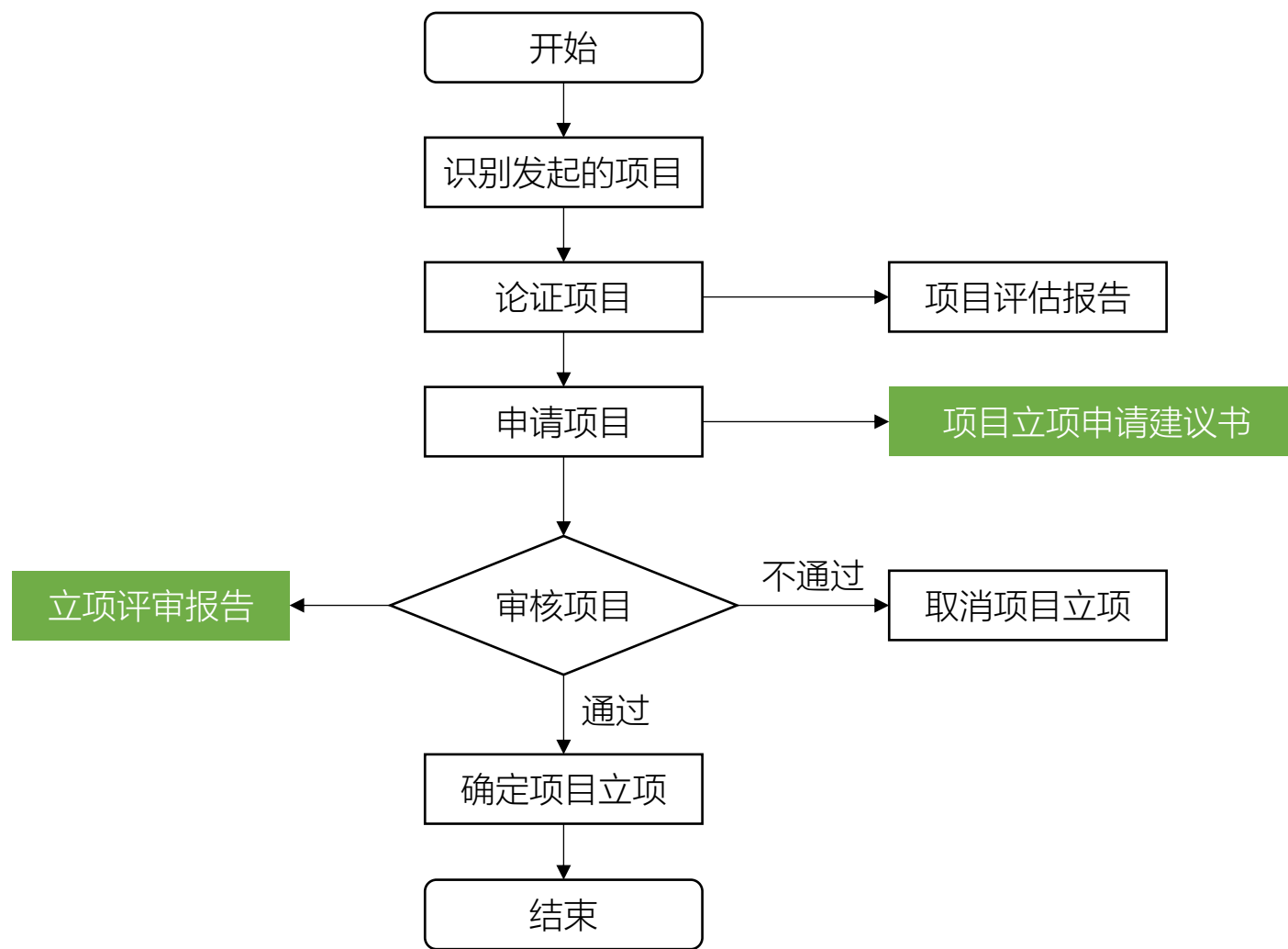
NPV

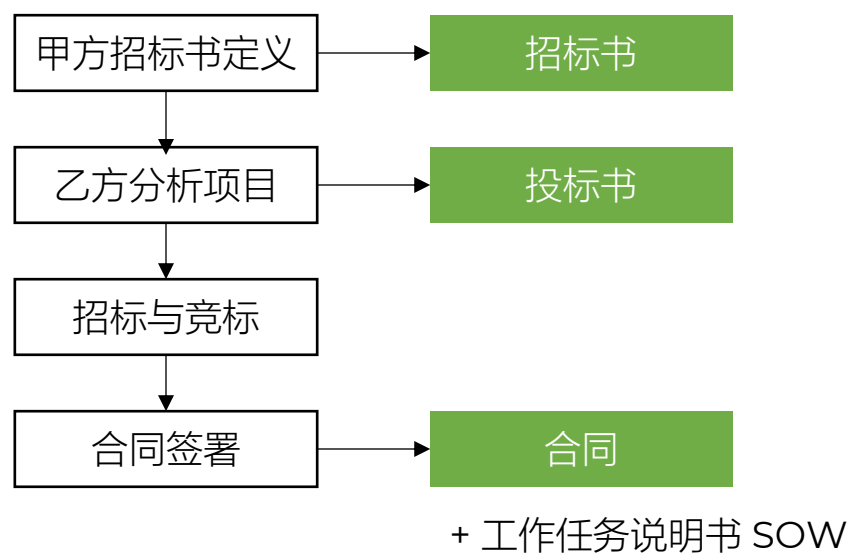
- 例2. 某软件公司准备投标某项目，项目的合同金额100万，分2次支付，项目开始支付50万，项目结束50万，项目周期2年，贴现率为8%，求NPV。

t	Input流入	Output流出	Input-Output净值	NPVt
t0	50	0	50	50
t1	0	0	0	0
t2	50	0	50	$50/(1+8\%)^2$

$$NPV = \sum_{t=0}^n \frac{CF_t}{(1+r)^t} = 50 + 0 + \frac{50}{(1+8\%)^2}$$

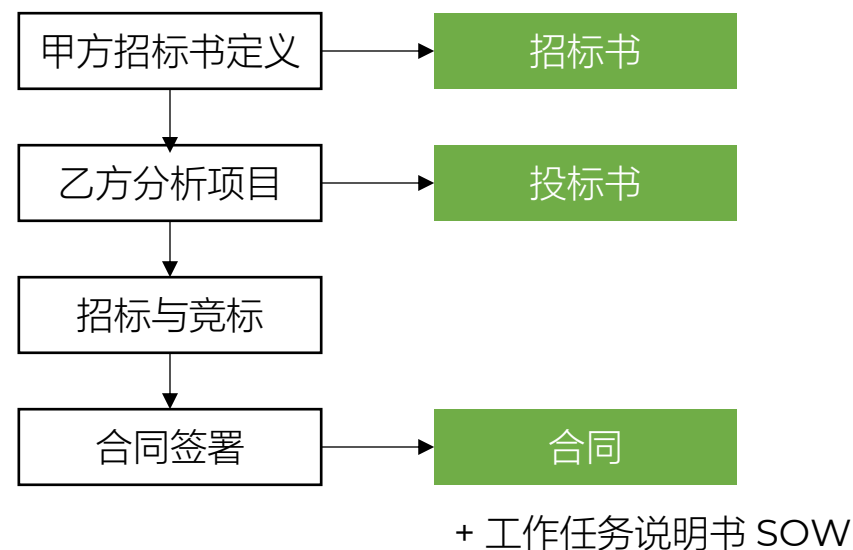






2.3.1 招标书

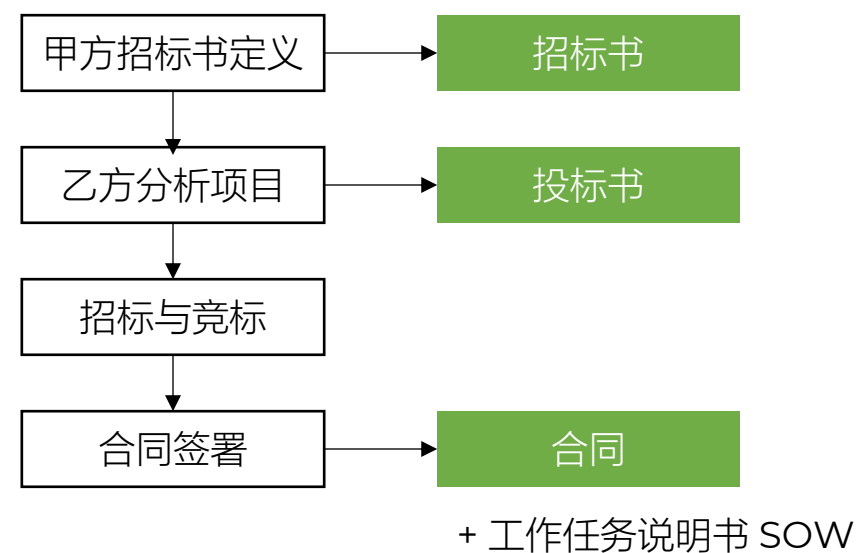
- IFB - Invitation for Bid (投标邀请)
 - 最正式, 全面考察
- RFP - Request for Proposal (建议书提交邀请)
 - 看看你的方案
- RFQ - Request for Quotation (报价邀请)
 - 看看你的报价
- IFN – Invitation for Negotiation (谈判邀请)
 - 非正式



2.3.2 投标书

Bid Proposal

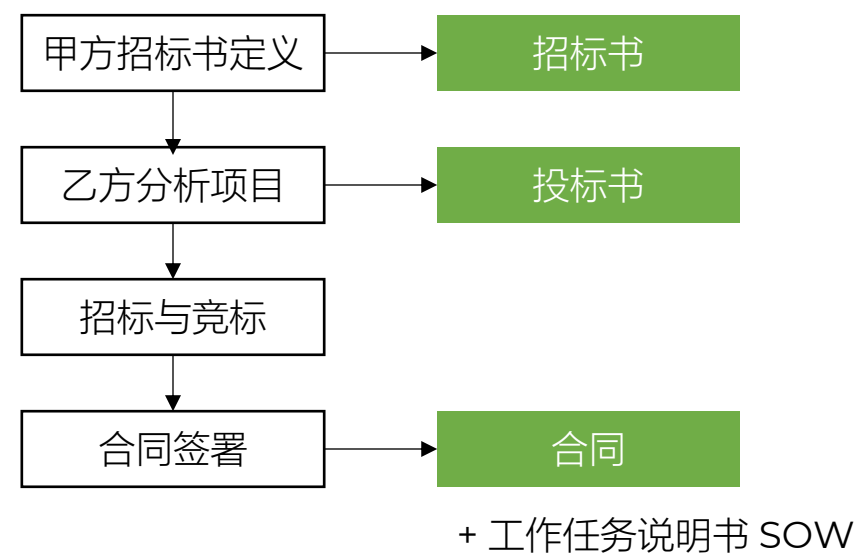
- 建议书 Proposal
- 报价单 Quotation



2.3.3 合同

Contract

- 更好的管理和约束双方的权力和义务



2.4 项目章程

Charter

- 指项目执行组织高层批准的一份以书面签署的确认项目存在的文件，包括对项目的确认、对项目经理的授权和项目目标的概述
 - 为项目提供合法性基础：正式文档；正式批准项目启动
 - 谁签发谁修改：项目发起人、出资人、高层管理签发
 - 不能频繁修改
- 说明
 - 不要写成论文：控制在 2~5 页内，重点突出目标、授权、范围、预算
 - 保持相对稳定：如需重大变更（如目标调整、预算翻倍），必须重新走审批流程



2.4 项目章程

Charter

- 项目名称
- 项目发起人及联系方式
- 项目经理及联系方式
- 项目目标
- 业务范围
- 工作描述/ 主要交付物概述
- 可交付成果（明确列出最终产出物）
- 时间安排/ 高层级里程碑计划（非详细进度，而是关键节点）
- 项目预算
- 成员/ 核心团队角色与职责概要
- 项目审批要求（谁有权批准变更？验收标准是什么？）
- 项目退出标准 / 成功标准（如何判断项目成功？）
- 主要风险摘要（高层级风险识别，如市场、技术、资源等）
- 假设条件与制约因素
- 项目授权声明（正式任命项目经理，赋予其动用组织资源的权力）
- 干系人清单（初步）（关键内外部干系人及其期望/影响力）
- 参考文件 / 依据（如合同、可行性研究报告、战略规划等）
- 供应商 / 外部合作方信息



2.4.1 项目章程 - 风险摘要

Charter

- 高层级、战略性风险
- 可能直接导致项目失败或严重偏离目标的重大威胁或重大机会
- 不是详细的风险分析或计划
- 作用
 - 预警机制：提醒发起人和管理层，这个项目不是稳赢的，存在哪些致命隐患
 - 决策依据：帮助发起人判断是否值得批准该项目（例如：技术风险太高，是否还要投钱？）
 - 资源预留：为后续制定应急储备（Contingency Reserve）提供初步依据

Items	Sample
技术风险 Technology	尚未在大规模数据下验证，过渡依赖热门AI，存在性能不达标风险。
市场/商业风险 Market	竞争对手可能在项目上线前3个月发布同类低价产品，导致市场份额预期下降50%。
资源风险 Resource	核心架构师未及时到位，可能导致前期设计延期。
合规/法律风险 Compliance	新出台的数据隐私法规可能导致当前设计方案需重新调整。
外部依赖风险 Dependence	项目依赖第三方供应商的API接口，若对方延期交付，整个系统将无法联调。



2.4.1 项目章程 - 风险摘要

Charter

- 更多合规/法律风险 Compliance

场景	风险描述示例	后果
数据隐私	个人信息保护法升级，导致当前用户数据采集方案违规	必须重构系统，否则面临巨额罚款
行业准入	医疗项目未在预计时间内获得药监局(NMPA)的临床试验批件	项目暂停，无法进入下一阶段
环保法规	新环保法禁止在特定区域施工，而项目选址恰好在此	选址变更或停工整改
劳工法	项目计划大量使用外包人员，但新劳动法限制了外包比例	人力成本激增或招聘计划受阻
知识产权	竞品公司起诉我方技术侵犯其专利	产品禁售或支付高额赔偿金
进出口管制	项目依赖的关键芯片被列入出口管制清单，无法进口	供应链断裂，硬件无法生产



2.4.2 项目章程 - 假设条件与制约因素

Charter

- 假设条件
 - 定义：制定计划时，被认为是真实、实际或确定的因素，但实际上并未得到证实。
 - 本质：我们赌它会是这样。如果假设错误，就会变成风险。
- 制约因素
 - 定义：限制项目团队行动的强制性限制。团队必须在这些框框内工作，没有商量余地。
 - 本质：“我们必须在这个范围内做”。

	假设条件 Assumptions	制约因素 Constraints
性质	我们认为是真的（但不确定）	必须遵守的限制（确定的）
如果出错	会变成风险 (Risk)	会导致违规或项目失败
灵活性	相对较高，可随信息明确而修正	极低，通常由高层或合同强制规定
口头禅	我们假定...	我们必须..." / "不能超过..."
例子	假定雨季不会影响室外施工。	必须在雨季结束前完工。



2.4.2 项目章程 – 示例

Charter

- 主要风险 Key Risks
 - 关键资源依赖：核心架构师到位延迟，存在前期设计延期及资源闲置风险。
 - 合规不确定性：新数据隐私法规可能导致技术方案返工。
 - 市场窗口：若无法在年底前上线，将错失春节营销高峰。



2.4.3 项目经理

Project manager

- 职责
 - 开发计划
 - 组织实施
 - 项目控制



2.4.3.1 项目经理

Project manager

- PMI人才三角 - PMI Talent Triangle
- 三者同等重要、缺一不可

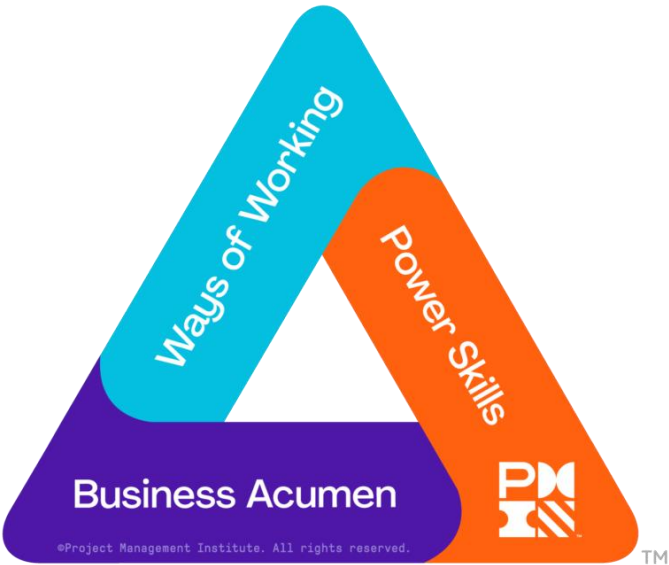


Item	Description
技术项目管理 Technical Project Management	与项目、项目集和项目组合管理特定领域相关的知识、技能和行为 即执行你岗位/角色所需的技术性方面
领导力 Leadership	涉及引导、激励和/或指导他人以实现目标的知识、技能和行为
战略与商业管理 Strategic and Business Management	对行业/组织的知识与专长，帮助你以增强绩效并更好交付业务成果的方式协调团队



2.4.3.2 项目经理

- PMI人才三角演进版



Item	Description
工作方式 Ways of Working	涵盖交付价值的方法、工具、技术和途径 - 无论是预测型、敏捷型、混合型还是其他新兴框架；不再局限于“项目管理”，而是关注“在任何情境下完成工作”
能力技能 Power Skills	指使个人能够影响、激励、协作和领导的人际能力 - 无论是否拥有正式职权。包括情绪智力、沟通、谈判、冲突解决和故事讲述等
商业洞察力 Business Acumen	指理解企业如何运作、盈利和创造价值的能力，并能运用这种理解来支持组织战略和财务健康的决策



2.4.3.3 项目经理

Project manager

- 标志着项目管理
 - 从“技术驱动”迈向“价值驱动”
 - 从“岗位能力”转向“通用胜任力”
- 最终目标
 - 培养能在复杂环境中持续创造商业影响力的复合型人才

Item	Former	later	Description
技术侧	Technical Project Management 技术项目管理	Ways of Working 工作方式	更强调方法论灵活性，而非仅限于项目管理本身
领导力侧	Leadership 领导力	Power Skills 权力技能 / 影响力技能	突出影响他人的核心能力
商业侧	Strategic and Business Management 战略与商业管理	Business Acumen 商业敏锐度	更简洁、聚焦于“理解业务并创造价值”的本质

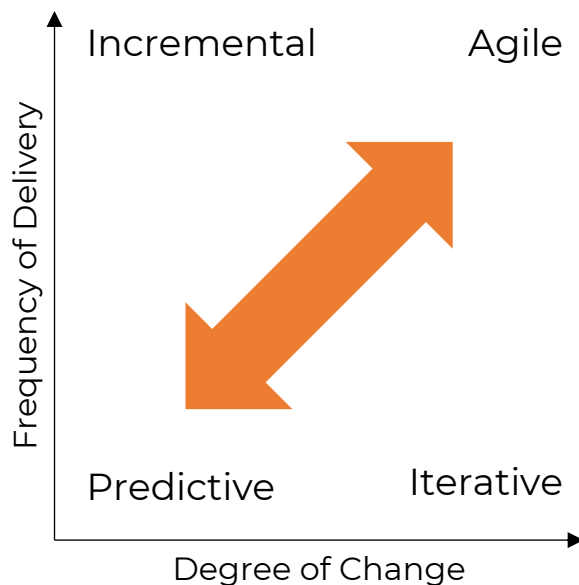


2.5 软件项目生存期模型

- 描述开发主要阶段
- 定义每个阶段要完成的主要过程和活动
- 规范每个阶段的输入和输出



2.5 软件项目生存期模型



方法	需求	活动	交付	目标
预测型 Predictive	固定	整个项目仅执行一次	一次交付	管理成本
迭代型 Iterative	动态	反复执行直至修正	一次交付	解决方案的正确性
增量型 Incremental	动态	对给定增量执行一次	频繁更小规模交付	速度
敏捷型 Agile	动态	反复执行直至修正	频繁小规模交付	通过频繁小规模交付和反馈实现客户价值



2.5.1 预测型

Predictive

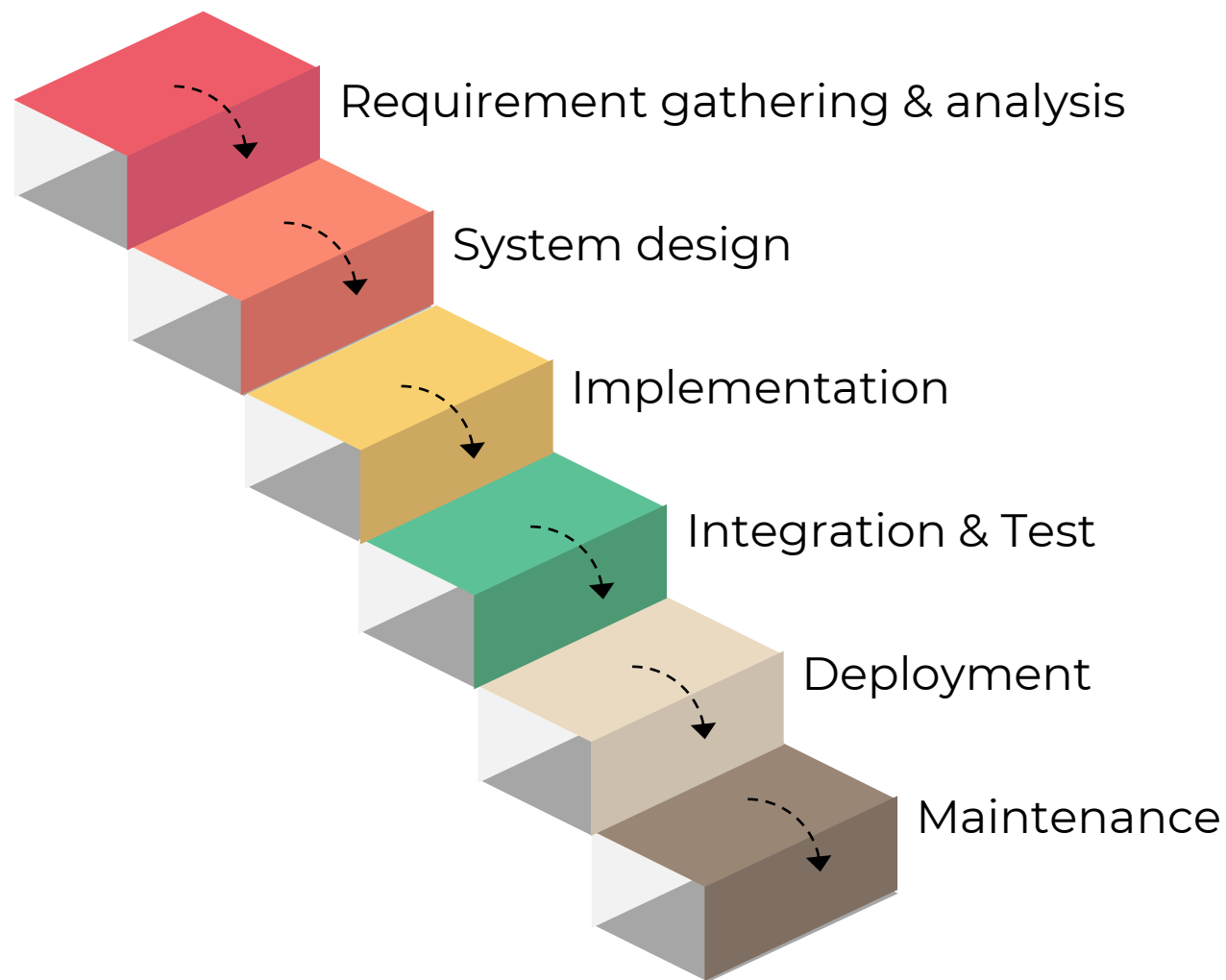
- 范围、进度、成本早期定义；对变化容忍度低
 - Scope, schedule, and cost defined early; low tolerance for change
- 特点
 - 需求清晰
 - 结构化
 - 在项目启动阶段就锁定范围、时间、成本（铁三角），后续严格执行
 - 变更被视为“异常”，需要走严格的变更控制流程
- 适用场景
 - 需求稳定、技术成熟、法规约束强的项目（如建筑、航天、医疗设备）
- 风险：
 - 如果前期判断错误，后期修正代价极高
 - 客户直到最后才能看到成果，容易货不对板



2.5.1.1 瀑布模型

Waterfall Mode /Predictive

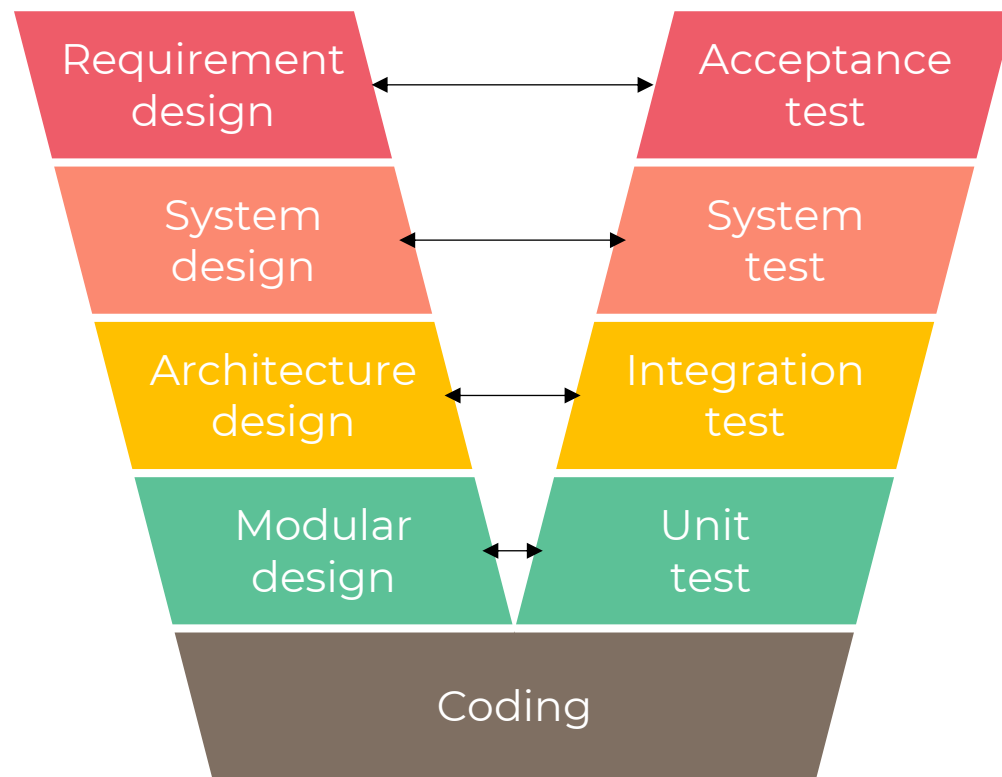
- Waterfall Model
- Winston Royce 于1970年提出
- 它将软件开发生命周期划分为若干线性阶段
- 每个阶段完成后才能进入下一个阶段
- 阶段分明、顺序执行、没有重叠
- 特点
 - 各阶段严格顺序、不可回溯（理想情况下）
 - 文档驱动，强调前期需求明确
 - 测试集中在开发后期
 - 不适合需求频繁变更的项目/返工



2.5.1.2 V模型

V Mode /Predictive

- Verification and Validation Model
- 瀑布模型的扩展和改进版本，强调测试与开发阶段的对应关系
- 其名称来源于图形呈现为V字形
 - 左侧是开发阶段
 - 右侧是对应的测试/验证阶段
- 特点
 - 测试计划在开发早期就制定
 - 强调尽早测试理念
 - 仍为线性模型，灵活性有限
- 适用
 - 需求明确、不能出错、且需要严格验证的项目
 - 质量保障体系模型



同步规划、并行设计、串行执行



方法	Waterfall Model	V Model
测试目的	检错	验证
测试计划	最后	同时
测试执行	最后	先后

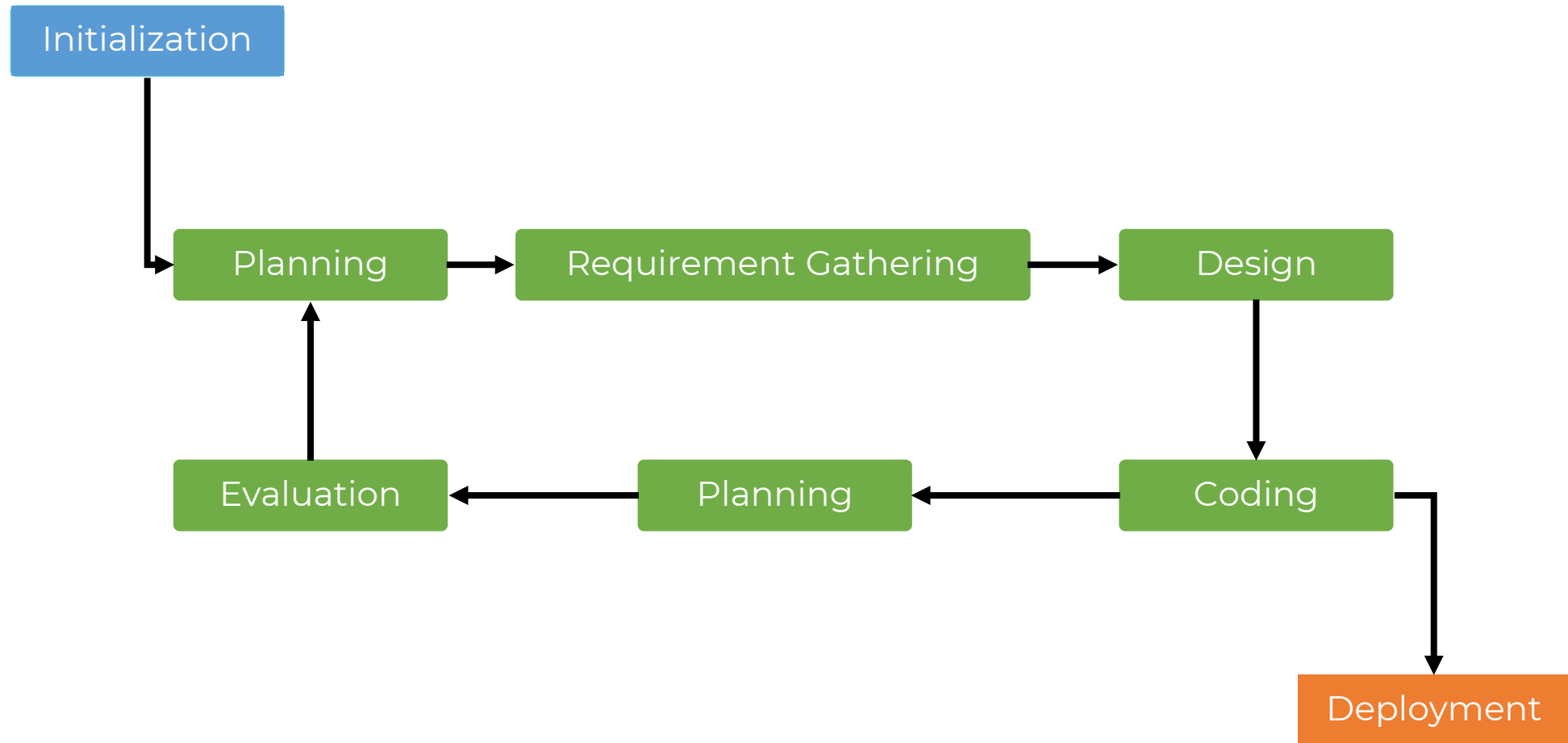


2.5.2 迭代型

Iterative Mode

- 通过迭代向基础软件产品中添加后续功能，直到最终系统完成
 - 产品通过循环逐步 refinement
 - 每次迭代改进设计或理解
 - Product refined over cycles; each iteration improves design/understanding
- 不追求一次性做对，而是通过多次循环（迭代）不断优化设计方案或对问题的理解
 - 也被称为循环模型
- 典型例子
 - 科研实验
 - 算法优化
 - UI/UX 原型测试
- 局限
 - 虽然内部在进化，但外部用户看不到价值，缺乏反馈闭环





2.5.3 增量型

- 复杂的项目划分为更小、独立的模块，这些模块被称为增量
 - 每完成一块就交付一部分功能
 - 每个增量增加功能
 - Product delivered in usable chunks; each increment adds functionality
- 用户能尽早使用部分功能，团队也能获得早期反馈
 - 快速交付价值 → 速度
 - 持续评估、反馈和调整是这个模型的核心，能够迅速识别和纠正错误
- 典型例子
 - 软件模块化开发（先上线登录注册，再上线支付）
 - 房地产分期交房
 - 拼图
- 局限
 - 如果整体架构没想清楚，可能导致拼凑式系统，后期难以维护
 - 缺乏反复修正的机制，可能方向跑偏



2.5.3 增量型

Incremental Mode

- 过程
 - 需求收集 Requirements Gathering
 - 设计 Design
 - 实现 Implementation
 - 测试 Testing
 - 集成 Integration
 - 评估和反馈 Evaluation and Feedback
 - 迭代过程 Iterative Process



	迭代 (Iteration)	增量 (Incremental)
核心动作	重复 (Repeat)	增加 (Add)
关注点	质量 (Quality): 把现有的做得更好、更稳	范围 (Scope): 把功能做得更多、更全
产出物	一个更完善的版本 (功能可能没变多, 但更好用了)	一个功能更多的版本 (多了新模块、新按钮)
风险策略	通过早期原型降低理解风险 (是不是做错了?)	通过分批交付降低交付风险 (能不能按时做完?)
用户视角	哇, 这个功能比上次快了很多/好看了很多!	哇, 这次多了一个新功能!
英文词根	Iterate (重做)	Increment (增长)



建造房子的过程是什么模型？

例题
Sample



2.5.4 敏捷型

Agile

- 本质
 - 敏捷 = 迭代 × 增量 × 反馈 feedback
 - 它是一个自适应系统，而非固定流程
- 高响应变化；依赖干系人反馈
 - Combines iterative + incremental; high responsiveness to change; stakeholder feedback
- 不是简单叠加，而是有机融合：
 - 用迭代来应对不确定性（不断试错、调整方向）；
 - 用增量来确保价值流动（每个迭代都交付可用成果）；
 - 用反馈来驱动决策（客户/用户参与评审，决定下一步做什么）。
- 高响应变化是结果，频繁反馈是手段。
- 目标
 - 最大化客户价值
 - 通过频繁小规模交付和反馈实现的客户价值



2.5.4 敏捷型

- 常见应用框架

模型	核心关注点	节奏/周期	角色定义	最佳适用场景
Scrum 敏捷	迭代与增量	固定 Sprint (2-4周)	严格 (PO/SM/Team)	新产品开发, 需求多变
Kanban 看板	流动与效率	持续流动 (无固定周期)	灵活 (无特定角色)	运维、支持、持续交付
XP 极限编程	工程质量	通常配合 Scrum 使用	灵活 (强调结对)	高复杂度、高质量要求的代码开发
FDD 特性驱动	功能特性	按特性交付	较正式 (首席程序员等)	大型项目, 需明确进度
Crystal 水晶	人与沟通	灵活	灵活	强调团队协作的文化建设



2.5.4.1 Scrum

- 源于橄榄球比赛中的争球（Scrum）
- 在橄榄球中，争球是比赛暂停后重新开始的方式：双方前锋队员紧密围拢，低头抵肩，形成密集阵型，共同协作以争夺控球权
- 1986年被哈佛大学教授竹内弘高和野中郁次郎引入管理学领域。在《哈佛商业评论》发表的《新新产品开发游戏》一文中，用来比喻高效团队的工作方式：团队作为一个整体，在明确的目标下紧密协作、灵活应变、共同前进
- 这一比喻随后被杰夫·萨瑟兰等人借鉴，并融合了迭代、增量开发等思想
- 1990s，形成了现代的Scrum敏捷开发框架
- 其核心精神与橄榄球争球高度一致：强调团队的自组织、跨职能协作、快速响应变化，以及为了同一个目标而并肩努力
- Scrum一词不仅是一个名称，更形象地传达了其团队协作与敏捷的核心哲学



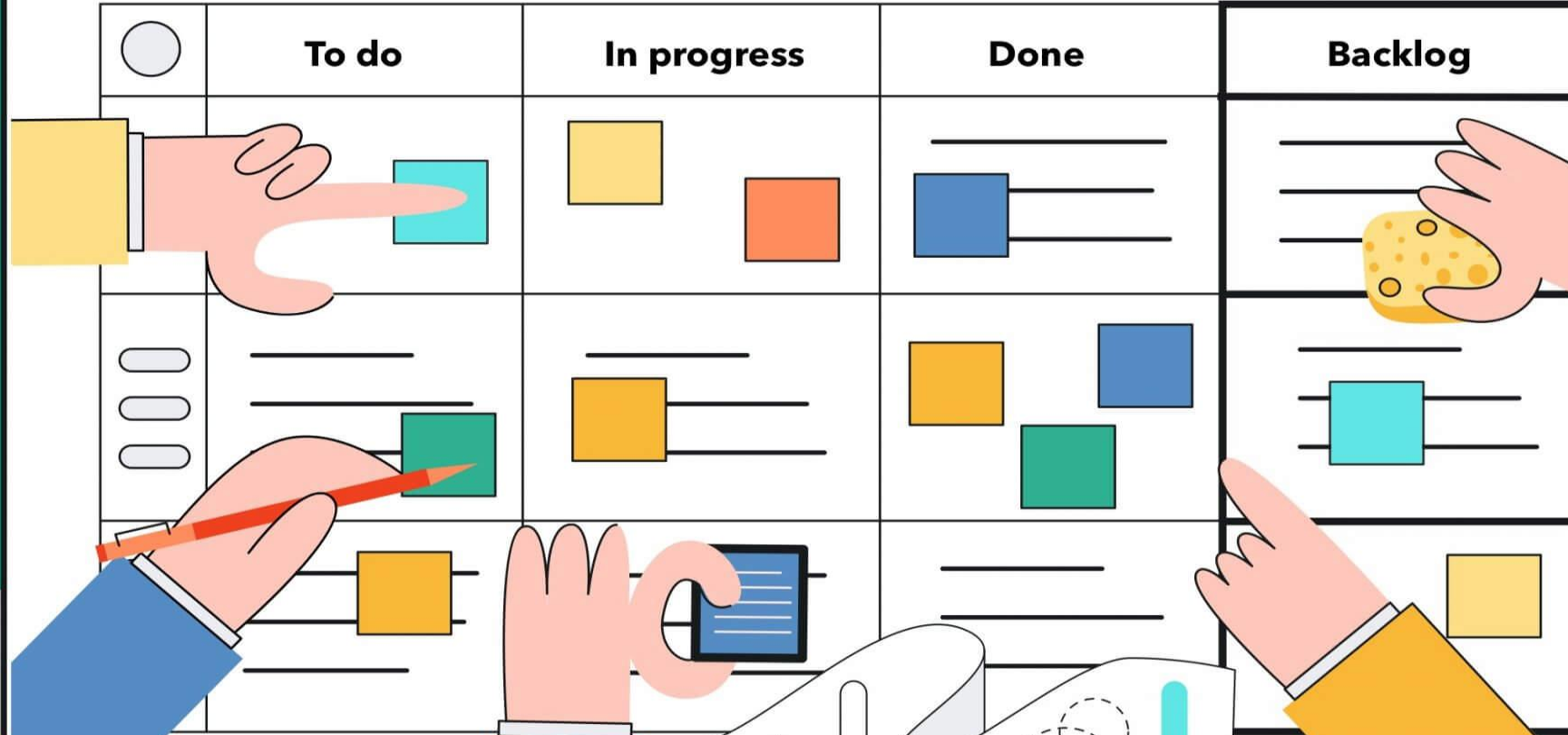
2.5.4.2 看板

- 看板是日语中的一个词
- 1940s, 日本丰田汽车公司提出
 - 为了管理制造过程而发明
 - 包含完成产品所需所有信息的卡片, 记录了产品从开始到完成的每个阶段
- 这个框架或方法在软件测试中广泛应用, 尤其是在敏捷方法中
- 强调持续改进和工作流程开放性的视觉项目管理模型
- 在软件测试中, 看板的主要作用是展示项目状态, 以便清楚地了解待办事项中等待被选择的工作, 正在进行的任务, 以及已完成的工作
- 在软件测试中, 看板有助于提高组织效率, 保持项目流程的连贯性, 并在整个过程中促进工作状态的透明度

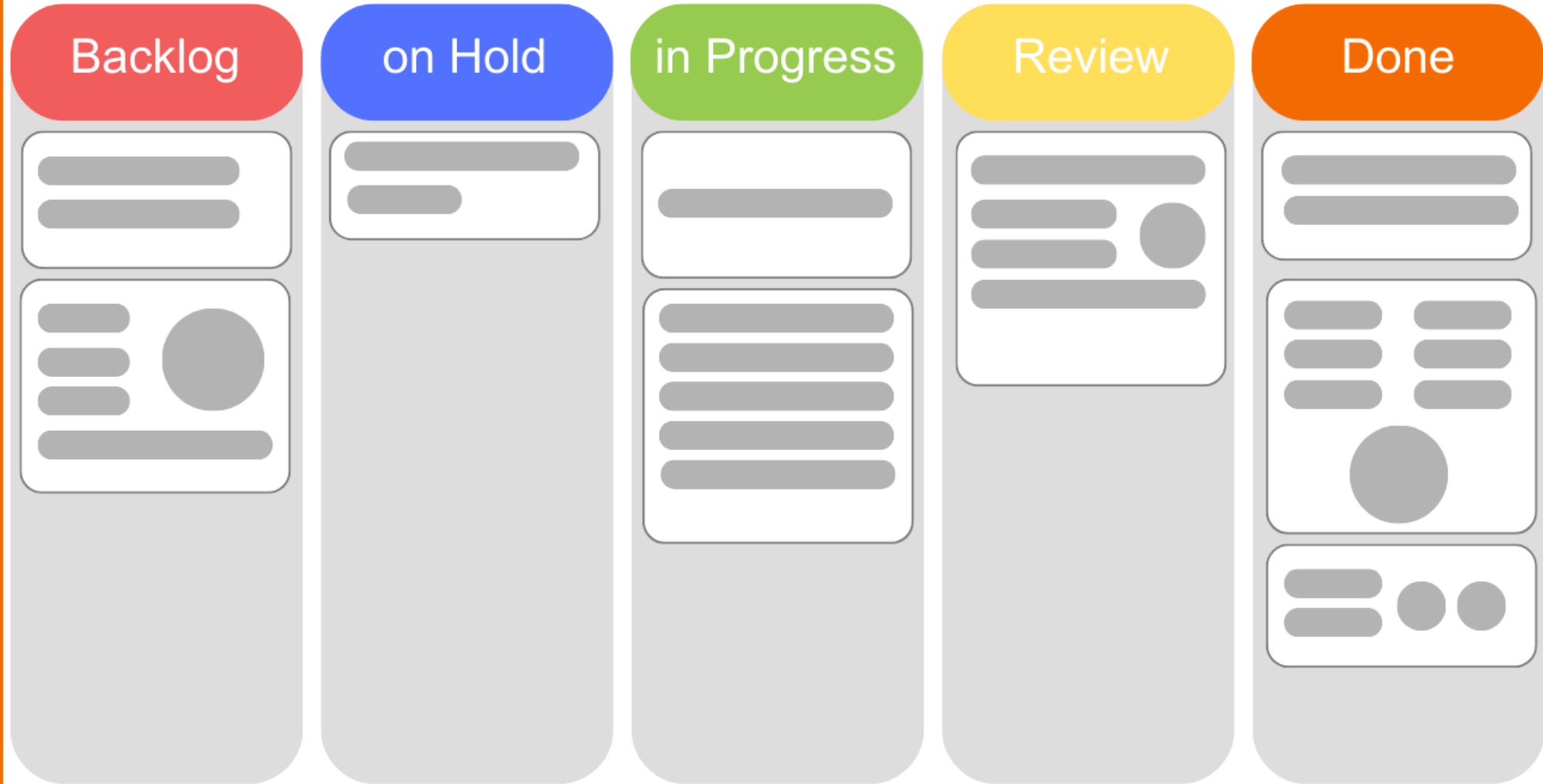


KANBAN CARDS

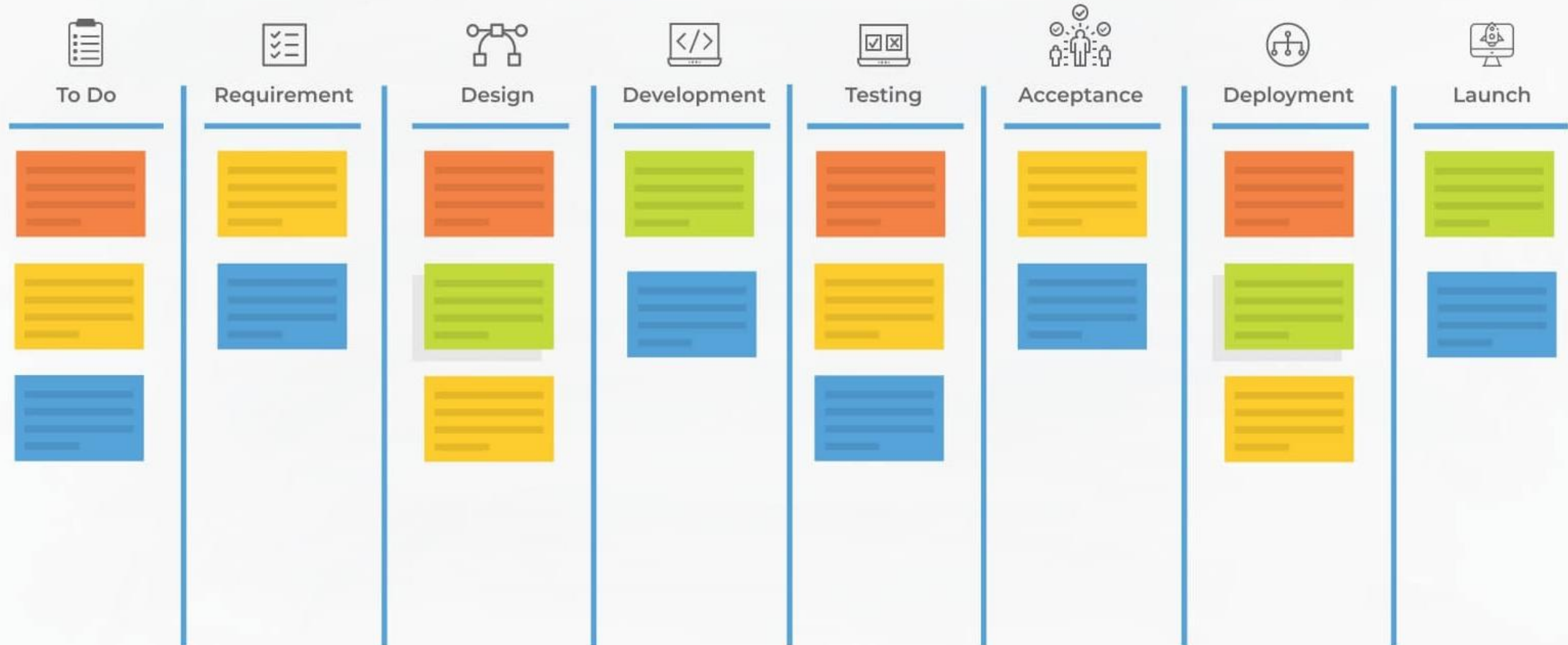
The illustration shows a Kanban board with four columns: 'To do', 'In progress', 'Done', and 'Backlog'. The board is a grid with three rows. The 'To do' column has a light blue circle icon in the top row. The 'In progress' column has a yellow square icon in the top row. The 'Done' column has a blue square icon in the top row. The 'Backlog' column has a yellow square icon in the top row. Hands are shown interacting with the board: a hand points to a cyan square in the 'To do' column; a hand holds a red pencil over a green square in the 'To do' column; a hand holds a blue card in the 'In progress' column; a hand holds a yellow card with a blue square in the 'Backlog' column; and a hand points to a yellow square in the 'Backlog' column. Below the board, there are two pieces of paper: one with a yellow triangle and dashed lines, and another with a blue circle and dashed lines.



Kanban-Board



Kanban Board



Column	Icon	Description	Role
To Do	 待办清单	尚未开始的工作项（如用户故事、Bug、技术债等）	输入池，等待被拉取进入流程
Requirement	 需求确认	需求已明确、可执行（可能包含验收标准）	“就绪”状态，确保下游不会因需求模糊而阻塞
Design	 系统设计	架构/界面/接口设计完成	防止开发中途返工，提升质量前置
Development	 编码实现	正在编写代码、单元测试	核心价值创造环节，通常 WIP 限制最严
Testing	 测试验证	QA 或自动化测试进行中	质量保证关卡，防止缺陷流入生产
Acceptance	 用户/PO 验收	产品负责人或客户确认功能符合预期	最终业务价值确认，避免“做错了事”
Deployment	 部署发布	自动或手动部署到预发/生产环境	技术就绪，准备对外服务
Launch	 正式上线	功能对用户可见、产生实际价值	价值交付终点，也是 Outcome 的起点



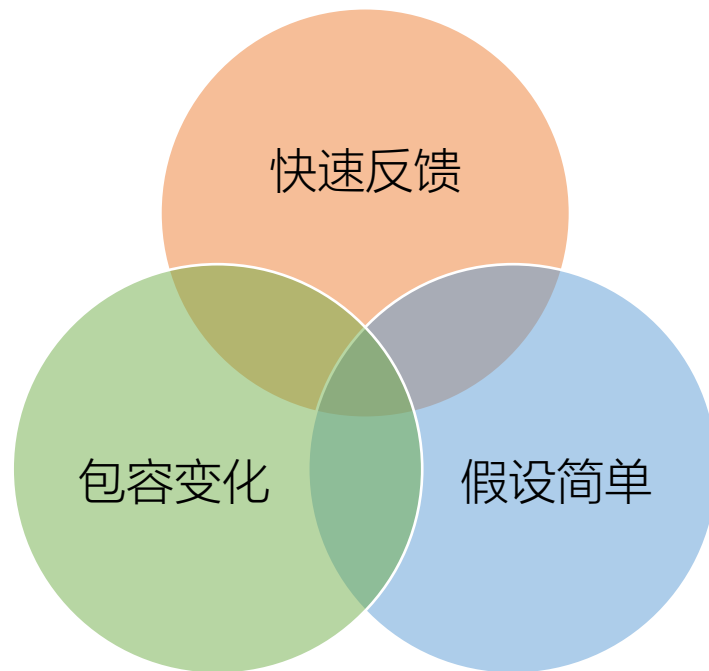
2.5.4.3 极限编程

- XP
- 敏捷开发家族中最具体、最严格、最具工程实践性的方法论
- 1990s, Kent Beck提出
 - 把软件开发中所有有益的做法, 都做到极致
- 适用场景: 需求不明确、变化极快、风险高、对代码质量要求高



2.5.4.3 极限编程

- 13个敏捷实践
 - 统一团队
 - 策划游戏
 - 小型发布
 - 客户验收
 - 代码集体所有
 - 编码标准
 - 恒定速率
 - 系统隐喻
 - 持续集成
 - 简单设计
 - 结对编程
 - 测试驱动开发
 - 重构



- 4大价值观 Values
 - 沟通 Communication: 面对面交流胜过文档。程序员必须结对, 必须和客户坐在一起
 - 简单 Simplicity: 只做当前需要做的事; 不要过度设计, 不要为未来可能的需求写代码
 - 反馈 Feedback: 快速获得反馈。通过单元测试、持续集成、短迭代, 立刻知道代码有没有错
 - 勇气 Courage: 敢于重构代码, 敢于说“不”, 敢于在发现错误时立刻停下来修复, 而不是掩盖
 - 尊重 Respect: +



2.5.4.4 特性驱动开发

- FDD
- 1990s, Jeff De Luca 和 Peter Coad 提出
 - 最初是为了解决大型银行系统（如新加坡大华银行）开发中遇到的复杂性和延期问题
- 主要目标是向客户提供及时更新且功能完善的软件
- 在FDD的所有级别，都需要进行报告和进度跟踪
- 设计和构建功能是此方法的核心
- FDD的生命周期：
 - 构建整个模型 - First of all entire model is built
 - 模型完成后，准备特征列表 - After the model is done, the feature list is prepared
 - 然后根据功能制定计划 - Then plan according to feature
 - 根据特性设计 - Design according to feature
 - 最后根据功能构建 - At last build according to feature



2.5.4.4 特性驱动开发

- **特性**不是模糊的需求，而是一个小的、客户可见的、有价值的功能单元
- 不是“模块”，不是任务，更不是技术实现，而是**业务价值的最小交付切片**
- 有一个严格的定义格式
 - 动作 Verb：描述系统要执行的操作（如：计算、验证、生成、记录）。
 - 结果 Result：描述操作产生的具体产出或状态变化。
 - 对象 Object：描述操作针对的具体业务实体（如：订单、用户、报告）。

<动作> + <结果> + <对象>
(Verb-Result-Object)



2.5.4.4 特性驱动开发

- 银行系统的“登录”功能

类型	描述	评价
✗ 错误	用户认证系统	太大，像是一个子系统
✗ 错误	编写登录界面的 HTML	技术任务，不是业务特性
✓ 正确	验证 (动作) 用户输入的密码 (结果/对象)	符合格式，可独立测试
✓ 正确	锁定 (动作) 账户在 (结果) 5 次失败尝试后 (对象/条件)	具体的业务规则
✓ 正确	生成 (动作) 临时登录会话令牌 (结果)	明确的技术业务价值



2.5.4.5 水晶

Crystal

- 1990s, Listair Cockburn提出
- 方法论家族
- 是开发软件中最轻量级和灵活的方法之一
- 它由多个敏捷流程组成，包括清晰、水晶黄、水晶橙以及其他具有独特特征的方法
- 用水晶来比喻这个模型，有两层含义：
 - 透明性：像水晶一样透明，强调团队内部的沟通清晰、可见
 - 多面性：像水晶有多个切面一样，Crystal 家族有多种颜色（变体），针对不同大小的项目
- 核心理念
 - 方法和流程应该根据项目的规模 Size、团队人数和系统的关键性 Criticality 进行量身定制
 - 不能一刀切
- 强调：人及人与人之间的互动，比流程和工具更重要



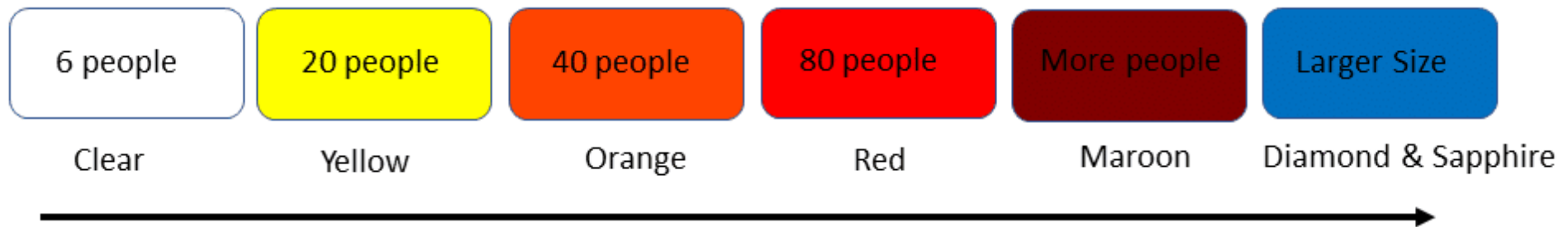
2.5.4.5 水晶

Crystal

- 根据团队人数和系统失败后的后果（关键性），将方法分为不同的颜色
- 团队越大、系统越关键，流程就越重（颜色越深）

颜色	适用团队规模	关键性 Criticality	特点描述
Crystal Clear 透明水晶	1 ~ 8 人	低 (舒适型)	最轻量级。几乎不需要文档，靠面对面沟通 适合小工具、内部系统
Crystal Yellow 黄水晶	10 ~ 20 人	中 (重要型)	需要少量的文档和定期的里程碑检查 适合一般商业应用
Crystal Orange 橙水晶	20 ~ 50 人	高 (关键型)	需要更正式的架构设计、详细的文档和严格的测试 适合银行、医疗系统
Crystal Red 红水晶	50 ~ 100 人	极高 (生命攸关)	非常严格，接近传统瀑布模型的严谨性，但保留敏捷迭代 适合航空控制、核电系统
...	更大	...	还有 Diamond (钻石) 等更重的变体 用于超大型项目





	Clear	Yellow	Orange	Red	Maroon
Life (L)	L6	L20	L40	L80	L200
Essential Money (E)	E6	E20	E40	E80	E200
Discretionary Money (D)	D6	D20	D40	D80	D200
Comfort (C)	C6	C20	C40	C80	C200
	1-6	7-20	21-40	41-80	81-200



	Scrum	XP	Crystal
核心关注点	流程框架 (角色、会议、工件)	工程实践 (结对编程、TDD、重构)	人与沟通 (根据情境调整)
灵活性	中等 (规则相对固定)	低 (必须遵守严格的技术实践)	极高 (没有固定规则, 按需定制)
适用场景	大多数敏捷项目	对代码质量要求极高的项目	各种规模和关键性的项目
文档要求	适中 (Backlog, Burn-down)	低 (代码即文档)	动态调整 (Clear 几乎无文档, Red 大量文档)
口号	Empirical Process Control	Technical Excellence	People over Processes



- <https://scrumguides.org>
 - Scrum is a lightweight **framework** that helps people, teams and organizations generate value through adaptive solutions for complex problems.
 - Changing the core design or ideas of Scrum
 - leaving out elements
 - not following the rules of Scrum
- ↓
- covers up problems and limits the benefits of Scrum
 - potentially even rendering it useless.

2.6.1 敏捷理念 Scrum Theory

Scrum /Agile

- 透明性 Transparency
 - must be **visible** to those performing the work as well as those receiving the work
 - low transparency can lead to decisions that diminish value and increase risk
 - Inspection without transparency is misleading and wasteful.
- 检视 Inspection
 - must be inspected **frequently** and **diligently** to detect potentially undesirable variances or problems
- 适应 Adaptation
 - Case1: any aspects of a process **deviate** outside acceptable limits
 - Case2: the resulting product is **unacceptable**
 - Case3: expected to adapt **the moment** it learns anything new through inspection
 - becomes more difficult when the people involved are **not** empowered or self-managing



2.6.1 敏捷理念 Scrum Theory

Scrum /Agile

透明性 Transparency

- 所有与过程、工作、进展和挑战相关的信息必须对团队和干系人清晰可见
- 工作项（如 Product Backlog）的内容、状态和优先级必须公开
- 团队使用统一的术语（如完成定义 - Definition of Done）
- 进度通过可视化工具（如 Sprint Burndown Chart、看板）展示
- 目的：确保所有人基于相同事实做决策，避免误解和隐藏问题

检视 Inspection

- Scrum 团队定期检查产品、过程和计划，以识别偏差或改进机会
- 在固定时间点进行：
- Daily Scrum：每日检视进度与障碍
- Sprint Review：检视可交付成果是否满足目标
- Sprint Retrospective：检视流程与协作方式
- 检视必须及时（不能等到项目结束），且不影响正常 workflows
- 目的：尽早发现问题，防止小偏差演变成大风险

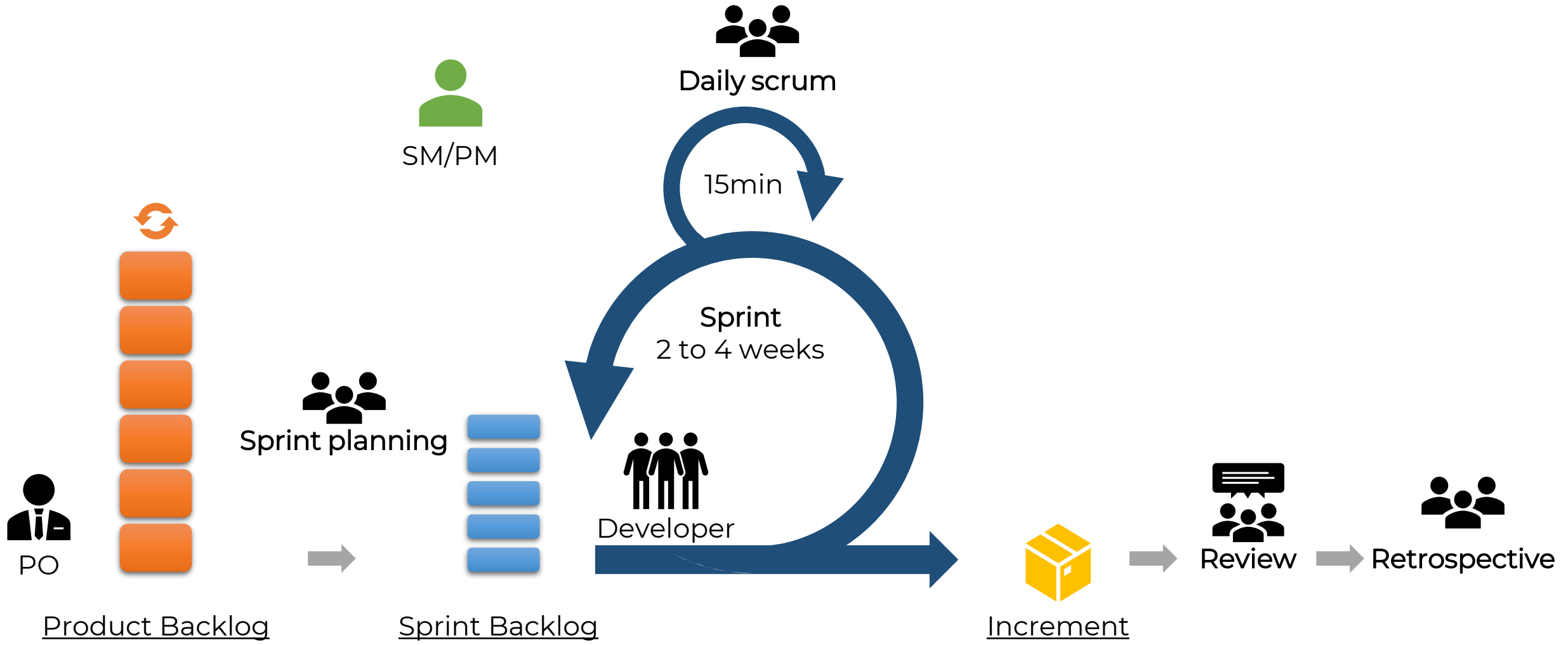
适应 Adaptation

- 基于检视结果，团队必须迅速调整行为、计划或方法
- 如果发现产品方向偏离，可在下个 Sprint 调整 Backlog
- 如果流程效率低，可在 Retrospective 后改进工作方式
- 适应必须在可接受的时间窗口内发生（通常在下一个 Sprint 开始前）
- 目的：持续优化交付价值的能力和团队效能



Items		Description
5	价值观 Values	承诺 Commitment 专注 Focus 开放 Openness 尊重 Respect 勇气 Courage
3	团队 Team	开发者 Developer 产品负责人 Product Owner 敏捷教练 Scrum Master
5	事件 Events	冲刺 The Sprint 冲刺规划 Sprint Planning 每日站会 Daily Scrum 冲刺评审 Sprint Review 冲刺回顾 Sprint Retrospective
3	工件 Aircrafts	产品待办事项 Product Backlog 迭代待办列表 Sprint Backlog 增加 Increment





2.6.3 敏捷价值观 Scrum Values

Scrum /Agile

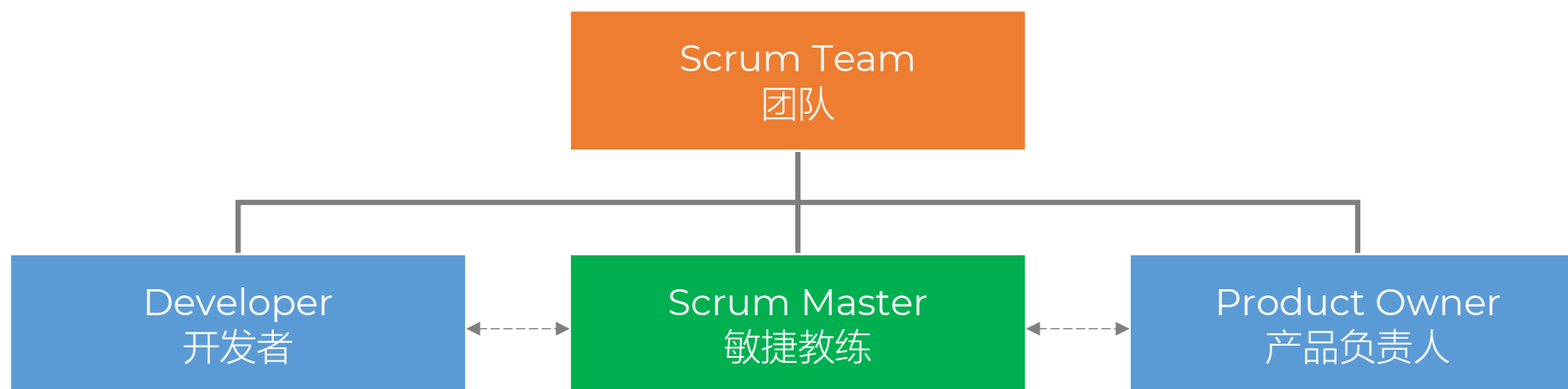
- 5个价值 values
- Successful use of Scrum depends on people becoming more proficient in living five values:
 - Commitment
 - Focus
 - Openness
 - Respect
 - Courage



2.6.4 敏捷团队 Scrum Team

Scrum /Agile

- 3个角色 Roles



2.6.4.1 开发者

Developer

- 制定计划，确定冲刺代办列表
 - Creating a plan for the Sprint, the Sprint Backlog
 - 根据完成定义，坚守质量标准
 - Instilling quality by adhering to a **Definition of Done (DOD)**
 - 根据目标调整计划
 - Adapting their plan each day toward the Sprint Goal
 - 作为专业人员，要互相负责
 - Holding each other accountable as professionals.
-
- 将冲刺目标转化为增量交付的任何方面的人员
 - 创造增量/价值
 - 不仅仅是单纯的开发团队
 - 5-9人



2.6.4.2 产品负责人

Product Owner

- 开发并明确地表达产品目标
 - **Developing** and **explicitly communicating** the Product Goal
- 创建并清晰地传达产品代办事项
 - Creating and **clearly** communicating Product Backlog items
- 对产品代办事项排序
 - Ordering Product Backlog items
- 确保产品代办事项透明、可见和可理解
 - Ensuring that the Product Backlog is transparent, visible and understood
- PO
- 是一个人，而不是一个委员会
- 可以委托其它人，但最终负责人还是他



2.6.4.3 敏捷教练

Scrum Master

- 也称项目经理 PM
- 真正的领导者：组建团队、主持冲刺会议、移除开发障碍
- 仆人式领导 a servant-leader
- 促成者 Facilitator
- 不是团队的保姆或超级英雄



2.6.4.3 敏捷教练

Scrum Master

- 作为教练在自管理和跨职能方面辅导团队成员
 - **Coaching** the team members in self-management and cross-functionality
- 帮助团队专注于创建符合 DOD 的高价值增量
 - Helping the Scrum Team focus on creating high-value Increments that meet the Definition of Done
- 促使移除团队工作进展中的障碍
 - **Causing** the removal of **impediments** to the Scrum Team's progress
- 确保所有 Scrum 事件都发生并且是积极的、富有成效的，并在冲刺时间盒内完成
 - Ensuring that all Scrum events take place and are positive, productive, and kept within the timebox

- 服务 Scrum Team



2.6.4.3 敏捷教练

Scrum Master

- 帮助找到有效定义产品目标和管理产品代办事项的技巧
 - Helping find techniques for effective Product Goal definition and Product Backlog management
- 帮助团队理解为何需要清晰且简明的产品代办事项
 - Helping the Scrum Team understand the need for clear and concise Product Backlog items
- 帮助建立针对复杂环境的基于经验主义的产品规划
 - Helping establish empirical product planning for a complex environment
- 当需要或被要求时，引导干系人之间的协作
 - Facilitating stakeholder collaboration as requested or needed

- 服务 Product Owner



2.6.4.3 敏捷教练

Scrum Master

- 带领、培训和作为教练辅导组织采纳 Scrum
 - Leading, training, and coaching the organization in its Scrum adoption
 - 在组织范围内规划并建议 Scrum 的实施
 - Planning and advising Scrum implementations within the organization
 - 帮助员工和干系人理解并实施针对复杂工作的经验主义方法
 - Helping employees and stakeholders understand and **enact** an empirical approach for complex work
 - 消除干系人和团队之间的隔阂
 - Removing barriers between stakeholders and Scrum Teams
- 服务 organization





2.6.5 敏捷活动

- 5个事件 Events



2.6.5.1 冲刺

Sprint

- No changes are made that would endanger the Sprint Goal
- Quality does not decrease
- The Product Backlog is refined as needed
- Scope may be clarified and renegotiated with the Product Owner as more is learned
- A Sprint could be cancelled if the Sprint Goal becomes **obsolete**
- Only the Product Owner has the authority to cancel the Sprint
- Tips
 - 自然月 Calendar Month vs 滚动月 Rolling Month



2.6.5.2 冲刺计划

Sprint planning

- Topics:
 - Topic One: Why is this Sprint valuable?
 - Topic Two: What can be Done this Sprint?
 - Topic Three: How will the chosen work get done?
- Sprint Planning is timeboxed to a **maximum** of eight hours for a one-month Sprint



2.6.5.3 每日站会

Daily scrum

- to inspect progress toward the Sprint Goal and adapt the Sprint Backlog as necessary, adjusting the upcoming planned work
- a 15-minute event
- at the same time and place every working day
- **improve** communications, **identify** impediments, **promote** quick decision-making, and consequently **eliminate** the need for other meetings
- often meet throughout the day



2.6.5.4 冲刺评审

- inspect the outcome of the Sprint and determine future adaptations
- presents the results of their work to key stakeholders
- progress toward the Product Goal is discussed
- The Product Backlog may also be adjusted to meet new opportunities
- avoid limiting it to a presentation
- the second to last event of the Sprint
- is timeboxed to a maximum of four hours for a one-month Sprint

2.6.5.5 冲刺回顾

- The Sprint Retrospective concludes the Sprint
- to plan ways to increase quality and effectiveness
- Discusses
 - what went well during the Sprint
 - what problems it encountered
 - how those problems were (or were not) solved
- identifies the most helpful changes ... for the next Sprint
- is timeboxed to a maximum of three hours for a one month Sprint



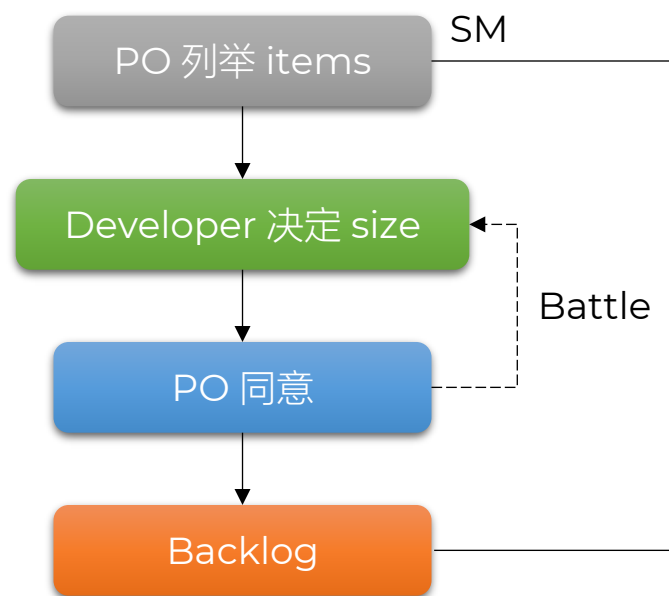
2.6.6 敏捷工件

- 3个工件 Aircrafts
- 代表工作或者价值



2.6.6.1 产品待办事项

- Product Backlog 产品待办事项
 - 包含产品目标 Product Goal 以及为实现该目标需要做的事情 items
 - 涌现 Emergent, 随着项目深入而逐渐清晰
 - 有序清单 ordered, 通常按价值、风险、必要性排序
 - Team 工作的单一 **single** 来源; 不在其中的事项, 可以不执行
 - PO 负责排序
 - Developer 决定每个 item 的规模 **Size**; PO 可以影响但不是命令
 - 一个 Sprint 内可以完成



2.6.6.2 冲刺待办事项

- 一份计划 Plan
- 组成 composition
 - Why: 实现冲刺目标 Sprint Goal
 - What: 从产品代办事项中选出的事项集合 Set
 - How: 为了实现交付需要制定可行 Actionable 的计划
- 在冲刺计划 Sprint planning 中确定
- 由 Developer 制定并实施 by and for; 体现自组织 Self-Management
- 彼此连贯 Coherence, 聚焦冲刺目标
- 随着认知的深入, 开发人员会随时对 Sprint Backlog 进行增、删、改, 以保持计划的真实性和有效性
- 工作偏离时, 需要和 PO 重新协商调整冲刺范围, 不能影响冲刺目标; 否则可能需要取消 Sprint 或重新规划



2.6.6.3 增量

- 为实现目标的坚实基础 concrete stepping stone
- 必须有用 Usable, 且经过检验 Verified, 价值累加 Additive
- 确保之前所有的增量能一起工作 Work Together
- 一个冲刺可以实现多个 Multiple 增量
- 完成定义 DOD(Definition of Done)
 - 只有符合完成定义 DOD, 才可被视作增量的一部分
 - 一旦一个 item 满足了完成定义, 一个增量就诞生了
 - 可以提前交付 prior to, 不用等到冲刺结束/不需要冲刺审核通过
 - 没有完成定义的 item 不能发布, 甚至不用在审查会议上展示, 但需要重新在产品代办事项中考虑
 - DOD 可以由组织制定, 所有团队遵守 – 最低标准 as a minimum
 - 也可以由某个团队制定并遵守 – 定制标准
 - 通常是2者兼有



2.7 开发流程

Process flow

- 每次敏捷迭代被称为冲刺
 - Each iteration of a scrum is termed as Sprint.
- 产品待办事项列表是包含创建最终产品所需所有信息的列表
 - A product **backlog** is a list that contains all of the information needed to create the final product.
- 每个迭代周期，从产品待办事项中挑选出顶级用户故事，并将其翻译到迭代待办事项中
 - The top user stories from the Product backlog are chosen and **translated** into Sprint backlogs during each Sprint.
- 团队处理已建立的冲刺待办事项
 - The team works on the sprint backlog that has been established.
- 团队会双重检查日常工作
 - The team double-checks the everyday job.
- 团队在冲刺结束时交付产品功能
 - The team delivers product functionality at the end of the sprint.



	Backlog	To Do
含义	所有潜在任务的总池	即将执行的任务子集
优先级	动态调整，无强制顺序	已排序，高优先级
粒度	可粗可细	通常已拆解为可执行单元
管理者	产品经理 / PO	团队共同确认
更新频率	持续添加/删除	每轮迭代前更新
类比	人生愿望清单	今日待办事项



维度	特性 Feature	增量 Increment
定义	单个、独立的业务功能单元	在原有软件基础上，新增功能后形成的新的、可运行的整体版本
视角	业务/功能视角（用户能用什么新功能？）	产品/版本视角（系统现在变成了什么样？）
构成	代码 + 测试 + 文档（针对单一功能）	所有已完成的特性 + 旧功能 + 修复的Bug + 性能优化
独立性	逻辑上独立，但可能依赖底层架构	必须独立可运行，不能依赖未完成的部分
累积性	不累积（它是一个点）	强累积（Increment 2 = Increment 1 + 新特性）
时间粒度	较小（通常几天到两周）	较大（通常是一个迭代/Sprint结束，或一个月发布一次）
FDD中的角色	FDD的核心驱动单位（按特性开发）	FDD的结果（每完成一组特性，就形成一个增量）





1. 需求管理
2. 传统项目需求分析
3. 敏捷项目需求分析
4. 任务分解
5. 敏捷任务分解

软件项目范围计划

Section 3 Project Scope

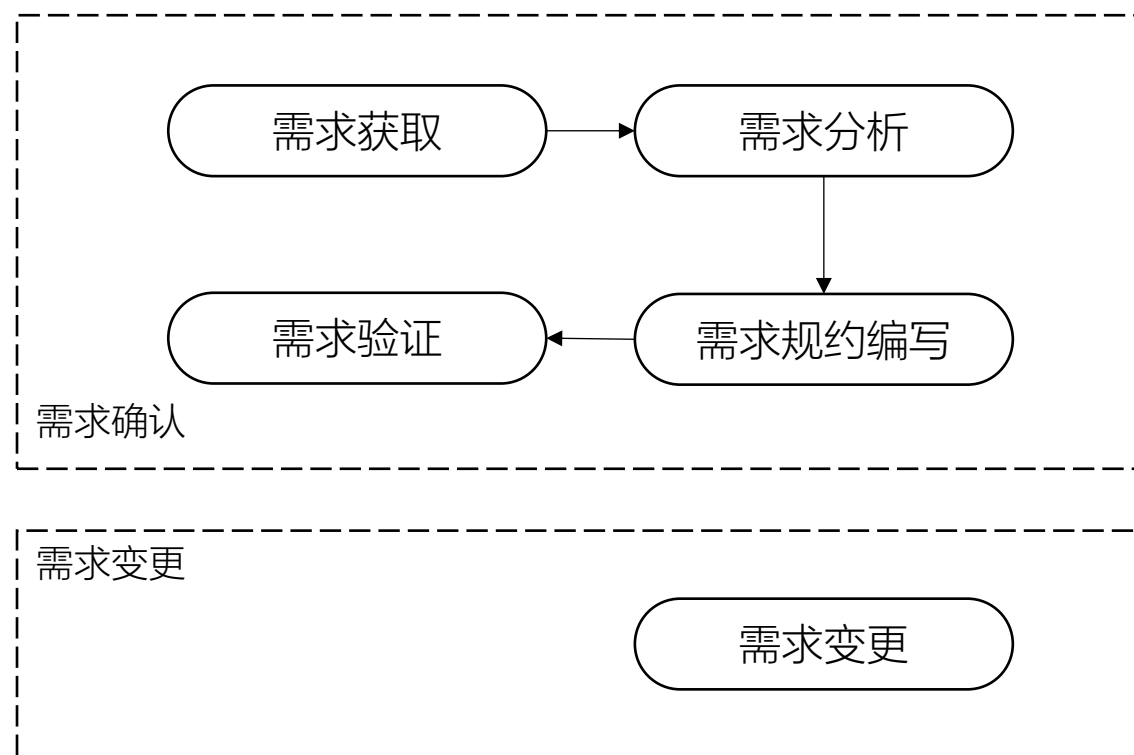


Rank	English	Chinese	Average
1	Inadequate requirements specification	不充分的需求规范	4.5
2	Changes in requirements	需求的改变	4.3
3	Shortage of systems engineers	缺乏系统工程师	4.2
4	Shortage of software managers	缺乏了解软件特性的经理人	4.1
5	Shortage of qualified project managers	缺乏合格的项目经理	4.1
6	Shortage of software engineers	缺乏软件工程师	3.9
7	Fixed-price contract	固定价合同	3.8
8	Inadequate communications for system integration	系统集成阶段，交流与沟通不充分	3.8
9	Insufficient experience as team	团队缺乏经验	3.6
10	Shortage of application domain experts	缺乏应用领域专家	3.6



1. 项目需求

- 是软件工程中将用户模糊的业务期望转化为清晰、可执行、可验证的系统规格说明的过程
- 它是连接“用户想要什么”与“开发团队做什么”的桥梁，直接决定了项目的成败
- 开发的基础
 - 做什么
 - 具备什么功能
 - 达到怎样的性能



- 分类
 - 业务需求 business requirement: 关注商业价值、战略目标、投资回报等/Why?
 - 用户需求 user requirement: C端的需求/What?
 - 功能需求 functional requirement: 必须具备哪些具体功能来满足用户需求/How?
- 需求获取
 - 访谈: 面对面沟通、电话、专题会议类
 - 文档: 问卷、数据筛选
- 需求分析
 - 也称系统建模
 - 为最终用户所看到的系统建立一个概念模型, 是对需求的抽象描述
 - DFD看数据, 流程图看步骤, 用例图看用户, 时序图看交互, E-R图看结构...
- 需求规约编写
 - 需求分析工作完成的一个基本标志
 - 一份正式的、完整的、规范的需求规格说明书
 - 基础性文档

- 需求验证
 - 正确性
 - 完整性
 - 一致性
 - 可行性：为了开发而开发
 - 必要性：是否影响核心价值和功能
 - 可检验性：可测试
 - 可跟踪性：流程可追踪；版本控制 git/GitHub
 - 签字确认

- 需求变更
 - 客观存在
 - 正视并接受需求变更
 - 由变更控制委员会SCCB(Software Change Control Board)审核并确定



2. 传统项目需求分析方法

Traditional

- 数据流图 DFD(Data Flow Diagram)
 - 4个元素：实体 External Entity、过程 Process、数据流 Data Flow、存储 Storage
- 系统流程图 System Flowchart
 - 使用标准流程图符号（开始/结束、处理、判断、输入/输出等
- 统一建模语言 UML(Unified Modeling Language)
 - E-R图, Entity-Relationship Diagram, 实体-关系图
 - 实体 Entity
 - 属性 Attribute
 - 关系 Relationship, 一对一、一对多、多对多
 - 用例图 Use Case Diagram
 - 参与者 Actor
 - 用例 Use Case
 - 关联 Association、包含 include、扩展 extend、泛化 generalization
 - 时序图 Sequence Diagram
 - ...



3.1 影响地图

Impact Mapping/Agile

- 影响地图 Impact Mapping



3.2 需求池

Product Backlog/Agile

- 需求池 Product Backlog
- 产品代办事项



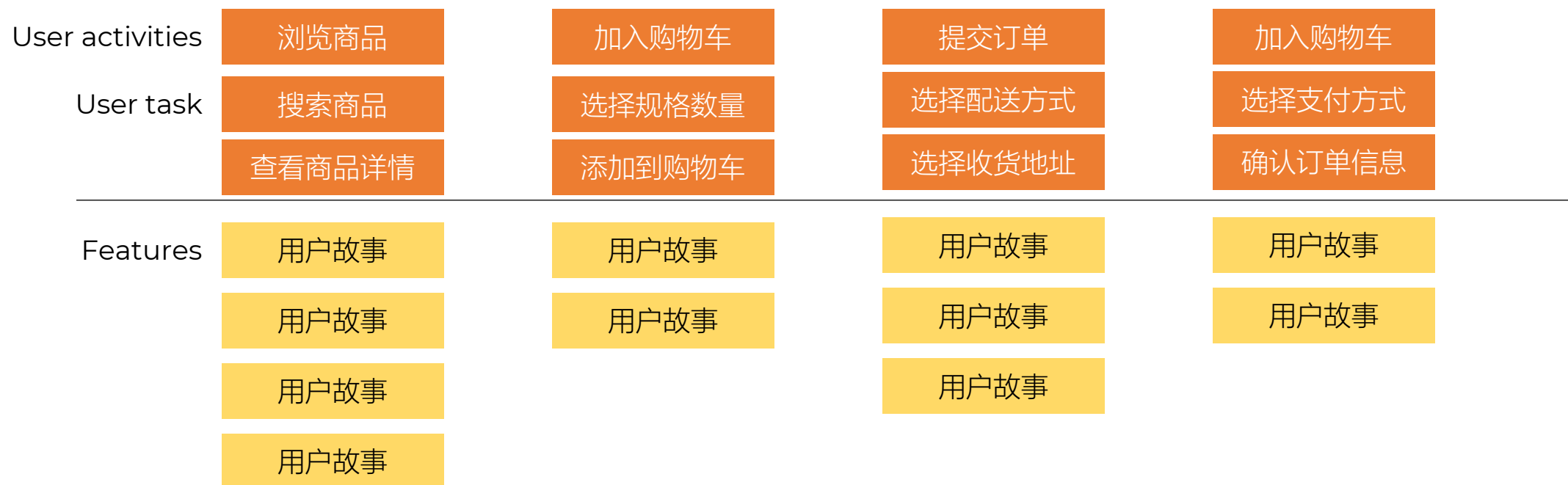
3.3 用户故事地图

- 用户故事地图 User Story Mapping
- 敏捷项目中，使用用户故事描述需求
 - In Agile process, user requirements are expressed as **user stories**.
- 将 backlog 变成一个二维图
 - 第1维度：一系列用户故事
 - 第2维度：用户故事的不同功能需求



3.3 用户故事地图

- 用户活动 User activities: 高层级的用户目标或阶段, 如注册、浏览商品、下单
- 用户任务 User task: 完成该活动所需的具体操作步骤
- 功能/用户故事 Features: 支撑每个任务的具体功能点, 通常按优先级纵向排列 (上高下低)



用户故事: 作为用户, 我想通过关键词搜索商品, 以便快速找到想要的商品。



3.4 用户故事编写

- 用户故事编写 User Story Writing

Story name		Story ID	
As a ... I want to... So that ...			
Priorities	High 1 2 3 4 5 low	Iteration number	
Date started		Date finished	

Acceptance Criteria
Notes:



3.5 INVEST原则

INVEST/Agile

- Independent 独立的
- Negotiable 可协商的
- Valuable 有价值的
- Estimable 可估算的
- Small 小的
- Testable 可测试的



3.6 行为驱动开发

BDD/Agile

- 行为驱动开发 BDD, Behavior-Driven Development



Construction Project Scope Creep Example

Acme Builders 已签订合同，承建一栋三层多户住宅建设已经开始，当客户找到总承包商时，表示为了使项目对他们作为房东有利可图，他们必须增加一层的建筑，并且要在同一时间内完成

例题
Sample



Manufacturing Project Scope Creep Example

A到B制造商正准备为新产品投入生产，他们已经设计并采购了手机壳的材料
执行团队未经监督地更改了设计，想添加一个夹子，让手机壳能够挂在用户的腰带上

例题
Sample



IT Project Scope Creep Example

三个月内，一位项目经理负责交付一款新的软件
在规划阶段几周后，项目的赞助商增加了新的功能。
在项目经理将新要求纳入项目范围后，赞助商根据项目客户的请求进行了更多更改

例题
Sample



2.4.2 定义项目范围

- 范围 Scope
 - 一份正式文件
 - 定义了完成项目所需的特定任务、可交付成果、时间表和责任
 - 在整个执行过程中也起到控制工具的作用
 - 当出现疑问或变更请求时，团队可以回顾工作范围，以确定请求是否在范围之内
 - 防止范围蔓延 **Scope Creep**
- 工作范围旨在
 - 定义项目可交付成果和边界 - Define project **deliverables** and **boundaries**
 - 明确角色、责任和期望 - Clarify roles, responsibilities and expectations
 - 建立时间表、里程碑和假设 - Establish timelines, milestones and assumptions
 - 降低风险、争议和计划外变更 - Reduce risk, disputes and unplanned changes



- 任务分解结构 - Work Breakdown Structure
- 将一个项目分解为更多的工作细目或者子项目，使项目变得更小、更易管理、更易操作
- WBS 组织并定义整个项目范围
- WBS 是对项目由粗到细的分解过程
- 面向交付成果的

- 实现目标的充分必要条件
- 一个清晰的任务完成
- 一个清晰的责任人
- 能够估算工期和工作量



- 图表式
- 清单式
- 字典



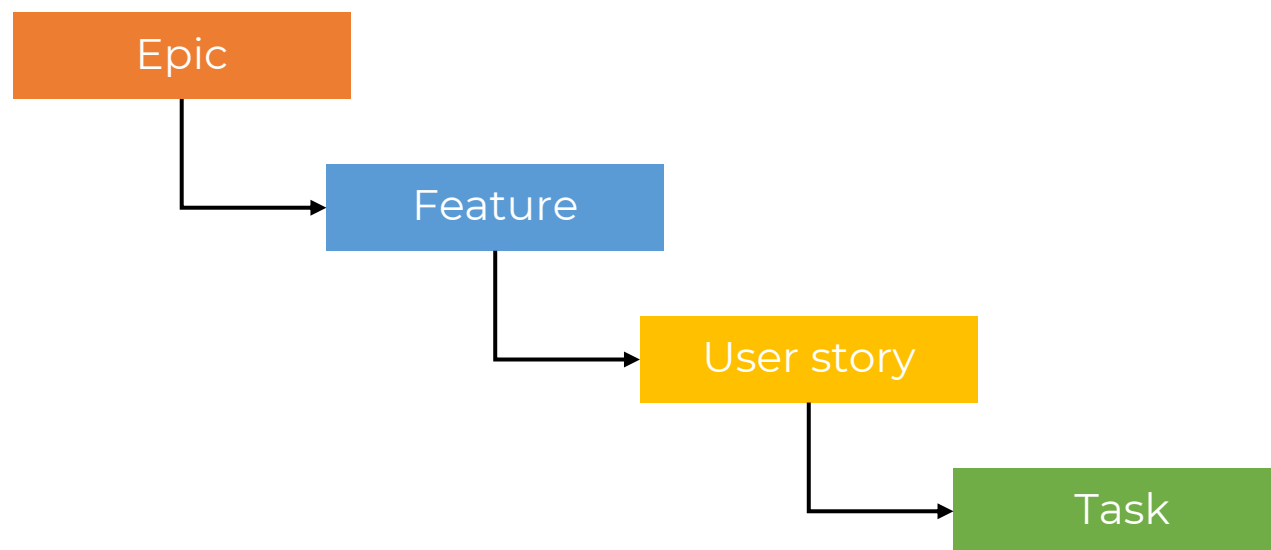
- 类比
- 模板参照
- 自上而下
- 自下而上



- 最低层是可控的和可管理的，但是不必要过细
- 每个 Work package 必须有一个提交物
- 定义任务完成的标准
- 有利于责任分配
- 推荐任务分解到 40 小时以内，敏捷项目分解到小时



- 基于用户故事 User Story 分解为具体的任务 Task
- 是冲刺计划会议 Sprint Planning 中的关键环节
- 有助于团队估算工作量、分配责任并跟踪进度
 - 好的拆分 = 按用户行为/功能模块划分
 - 每个故事应聚焦单一价值点
 - 优先交付核心功能，再逐步增强体验
 - 确保每个故事都满足 INVEST 原则



Epics

- User stories is that they can be written at varying levels of detail.
- Large user stories are generally known as epics.
- Because an epic is generally too large for an agile team to complete in one iteration, it is split into multiple smaller user stories before it is worked on.



Epic

As a busy executive,

I **want to** be able to save favorites on my mobile weather application
so that I can choose from a finite drop-down list to easily locate the weather in the destination I am travelling to.

Break down story 1

As a busy executive,

I **want to** be able to save a search location to my list of favorites from my mobile device
So that I can reuse that search on future visits.

Break down story 2

As a busy executive,

I **want to** be able to name my saved searches from my mobile device
so that I know how to access each saved search in the future.



Epic

As a busy executive,

I want to be able to save favorites on my mobile weather application so that I can choose from a finite drop-down list to easily locate the weather in the destination I am travelling to.

Break down story 3

As a busy executive,

I **want** the name of my saved searches to default to the city name, unless I choose to manually override it

So that I can streamline saving my searches.

Break down story 4

As a busy executive,

I **want** my “saved favorites” option on the user interface to be presented on a mobile device near the “search location”

So that I have the option of starting a new search or using a saved one, and can minimize my keystrokes within the weather application.



Acceptance Criteria

1. Acceptance Criteria is simply a high-level acceptance test that will be true after the agile user story is complete.
2. Normally written on the back of the story card.
3. It is a great way to ensure that a story is understood and to invite negotiation with the team about the business value that we are trying to create.



Story name		Story ID	
As a ... I want to... So that ...			
Priorities	High 1 2 3 4 5 low	Iteration number	
Date started		Date finished	

Acceptance Criteria
Notes:



- 用于系统性地拆解“净资产利润率”，即 ROE, Return on Equity
- 帮助企业管理者、投资者或分析师深入理解企业盈利能力的驱动因素
- 由美国杜邦公司（DuPont Corporation）在20世纪初开发的一种财务分析工具
- 核心思想是：将净资产收益率 ROE 分解为多个关键财务比率的乘积，从而揭示影响股东回报的根本原因
- 不是单一指标，而是一个层级式分解框架，从宏观到微观层层剖析企业的盈利模式、运营效率和资本结构



1. 概念和术语
2. 成本管理计划
3. 成本估算
4. 成本预算
5. 成本控制

软件项目成本计划

Section 4 Project Cost



1. 概念和术语 - 成本类型

- 可变成本 Variable Costs
- 固定成本 Fixed Costs
- 直接成本 Direct Costs
 - 人工费、工资
 - 差旅费（项目相关人员）
 - 材料费
 - 专用设备费
 - 特定工具
- 间接成本 Indirect Costs
 - 管理费
 - 办公租金、安保
 - 行政人员工资
 - 公司级税金
 - 通用设备租赁
- 机会成本
- 沉默成本



1. 概念和术语 - 应急储备

Project Cost

- Contingency Reserve



方法	定义	特点
类比估算	使用类似项目的实际数据进行估算	快速但精度低
自上而下估算	从整体项目开始，分解并分配资源	通常用于初步估算，由高层向下分配
自下而上估算	先估算每个工作包或活动的成本/工期，再 汇总到更高层级	精度高，基于详细信息
多方案分析估算	比较多种技术或方案的成本/收益	用于决策支持
参数估算	基于数学模型和历史数据（如每米成本） 进行估算	
专家判断	依靠专家经验进行判断	主观，不依赖数值区间
三点估算	使用乐观（O）、最可能（M）、悲观（P） 三个值，计算期望工期或成本	



4. 成本预算

- 总成本分摊到各个工作包
- 各个工作包分解到各个活动
- 成本基线：时间计划和成本预算计划

项目预算	管理储备	控制账户	应急储备		
	成本基准				
			工作包成本估算		
			活动成本估算		



5. 成本控制



5. 成本控制 – 挣值分析

- Earned Value Management
- 要素
 - 计划价值 PV
 - 挣值 EV
 - 实际成本 AC



5. 成本控制 – 挣值分析

Item	Description
PV, Planned Value	计划工作量的预算费用, 应该花多少
EV, Earned Value	已完成的实际工作量预算费用/价值
AC, Actual Cost	已完成工作量的实际费用, 实际花了多少
CV, Cost Variance	成本偏差 $CV = EV - AC$
SV, Schedule Variance	进度偏差 $SV = EV - PV$
CPI, Cost Performance Index	成本绩效指数 $CPI = EV / AC$
SPI, Schedule Performance Index	进度绩效指数 $SPI = EV / PV$
BAC, Budget at Completion	总预算即全部的 PV
ETC, Estimate to Complete	完工尚需估算 $ETC = EAC - AC$
EAC, Estimate at Completion	完工估算: 非典型偏差 $EAC = AC + (BAC - EV)$ 典型偏差 $EAC = AC + (BAC - EV)/CPI$
VAC, Variance at Completion	完工偏差 $VAC = BAC - EAC$





1. 规划进度
2. 定义活动
3. 排列活动顺序
4. 估算活动资源
5. 估算活动持续时间
6. 制定进度计划
7. 控制进度过程

软件项目进度计划

Section 5 Project Progress



- 规划进度
- 定义活动
- 排列活动顺序
- 估算活动资源
- 估算活动持续时间
- 制定进度计划
- 控制进度过程



- 要求
 - 正式或非正式
 - 详细或概括
- 产出
 - 项目进度管理计划

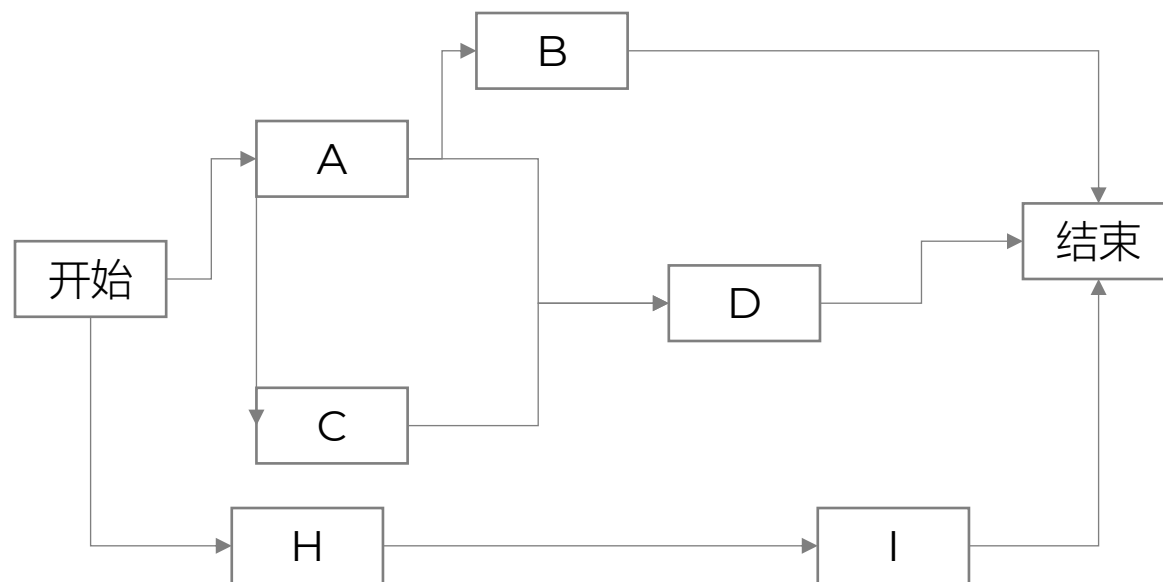


- 完成工作包所进行的工作
- 与工作包
 - 1 vs 1
 - M vs 1
- 排列活动的工具
 - 前导图 PDM, Precedence Diagramming Method, 也叫单代号网络图
 - 箭线图 ADM, Arrow Diagramming Method,, 也叫双代号网络图

前导图 PDM

- 活动：方块表示
- 紧前活动
 - H是I的紧前活动
- 紧后活动
 - I是H的紧后活动

- 结束 Finish → 开始 Start
 - FS: H结束了, I才可以开始
- 开始 Start → 开始 Start
 - SS: A开始, C也开始
- 开始 Start → 结束 Finish
- 结束 Finish → 结束 Finish



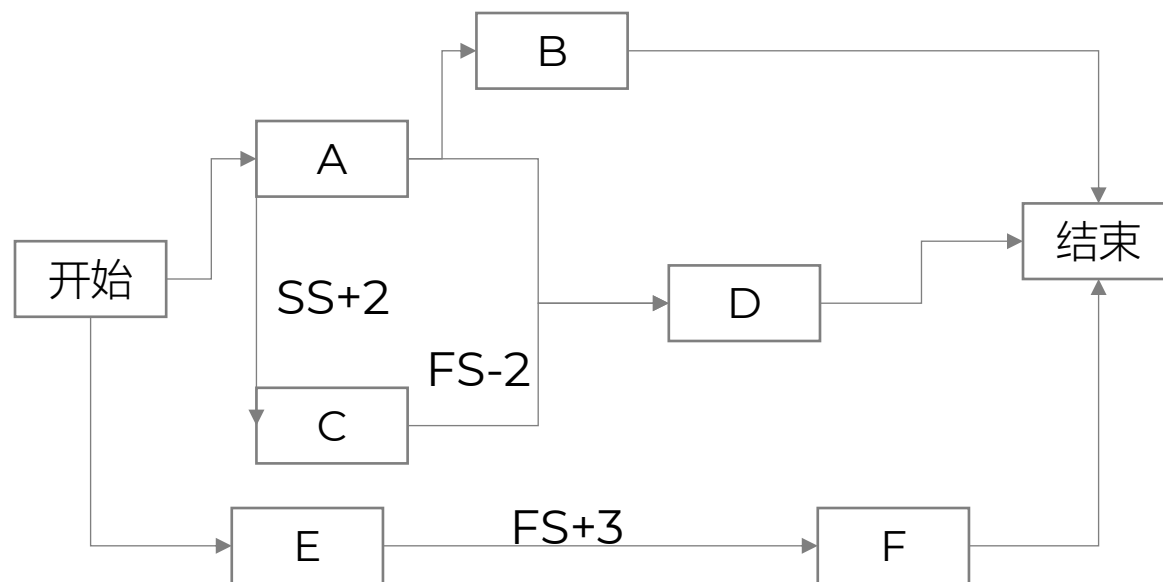
前导图 PDM

- 提前量 Lead

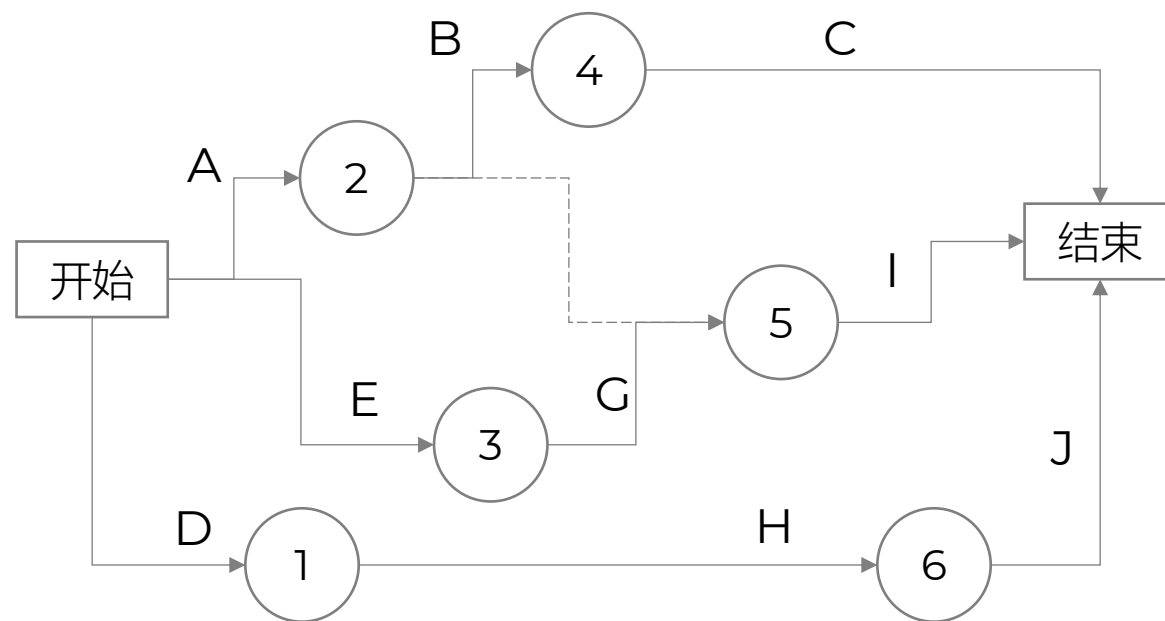
- 紧前活动还没有完成时，紧后活动可以提前开始的时间量
- 负值表示
- FS-2：完成到开始，提前2天，表示紧后活动可以在紧前活动完成前2天开始
- 谨慎使用

- 滞后量 Lag

- 紧前活动完成时，紧后活动需要推迟开始的时间量
- 正值表示
- FS+3：完成到开始，滞后3天，表示紧后活动必须在紧前活动完成后3天才开始



- 活动：以字母表示
- 圆圈：序号，以数值表示
- 可以使用虚拟活动 dummy activity 表示依赖关系



5. 估算活动持续时间

Project Progress

- 类比估算
- 三点估算



甘特图 Gantt Chart

- 甘特图是一种条形图，用于展示项目进度计划以及每个任务的开始时间和结束时间。它帮助团队成员了解项目的时间安排、任务之间的依赖关系以及整个项目的进展情况
- 组成
 - 事件号
 - 最早开始时刻
 - 最晚开始时刻
 - 时间持续时间
- 特点：
 - 显示每个任务的起止日期 - 进度的安排情况
 - 可以清晰地看到任务之间的重叠和依赖关系 - 体现并发任务
 - 帮助项目经理监控项目进度，并根据需要调整资源分配
 - 前续事件都完成才可以决定当前事件的执行时间
 - 最后一个的最早时间和最晚时间一样
 - 不能反映任务之间的依赖关系

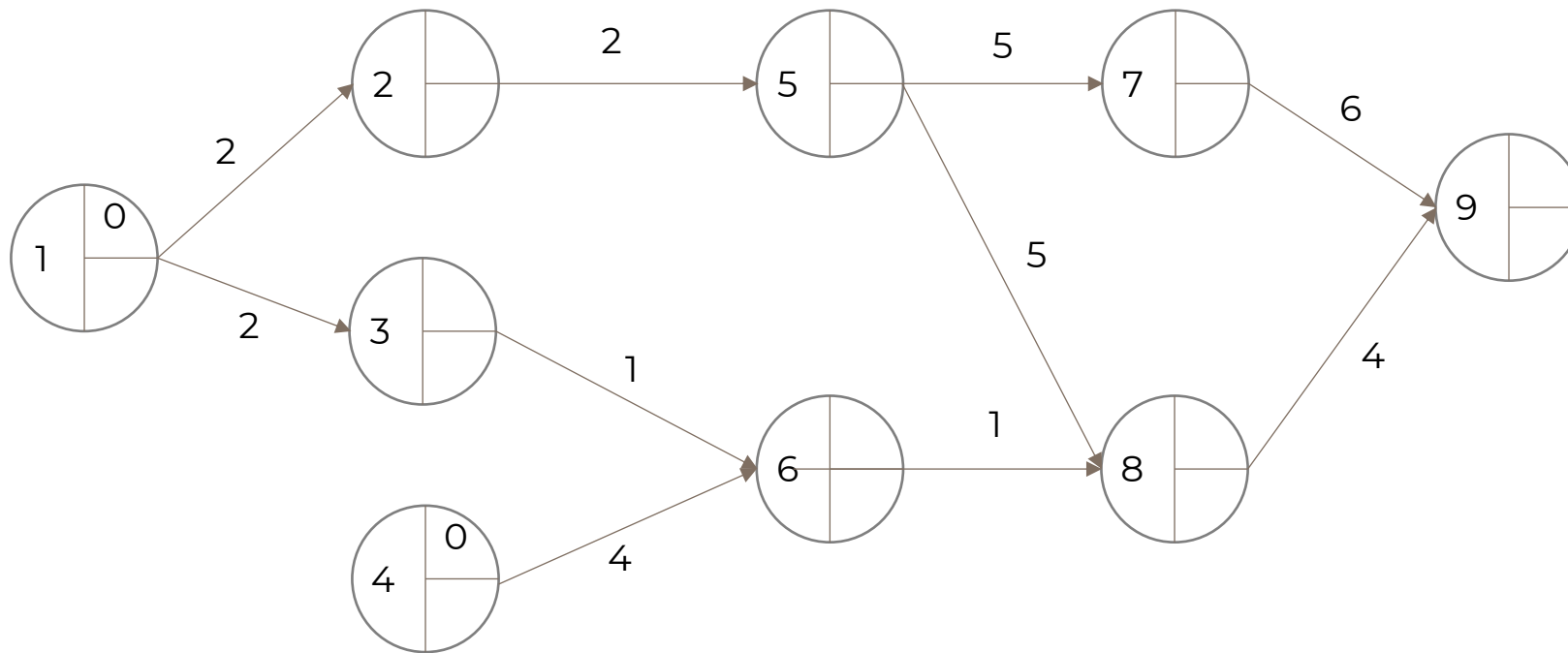


- Program Evaluation and Review Technique
- 在项目时间估算中的应用用于估算活动持续时间的统计工具，适用于不确定性较高的项目
- 组成：任务开始时间、结束时间、完成所需时间、任务之间的先后关系
- 特点：不能反映任务之间的并行关系
- 基于三个时间估计值 - 三点估算：
 - 乐观时间 (O)：最理想情况下的时间
 - 悲观时间 (P)：最坏情况下的时间
 - 最可能时间 (M)：正常情况下的时间

$$= \frac{O + P + 4M}{6}$$

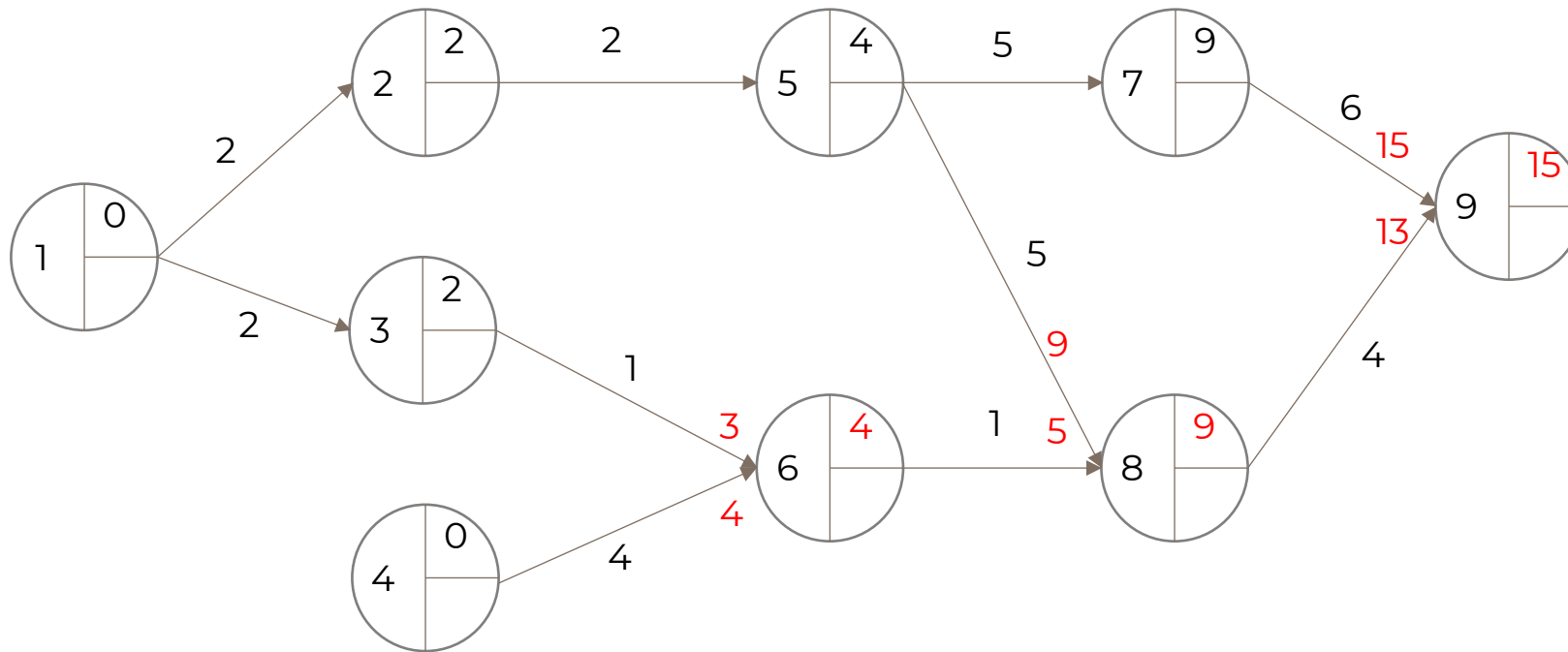
PERT

- 求各事件最早时间、最晚时间、总工期、最大路径



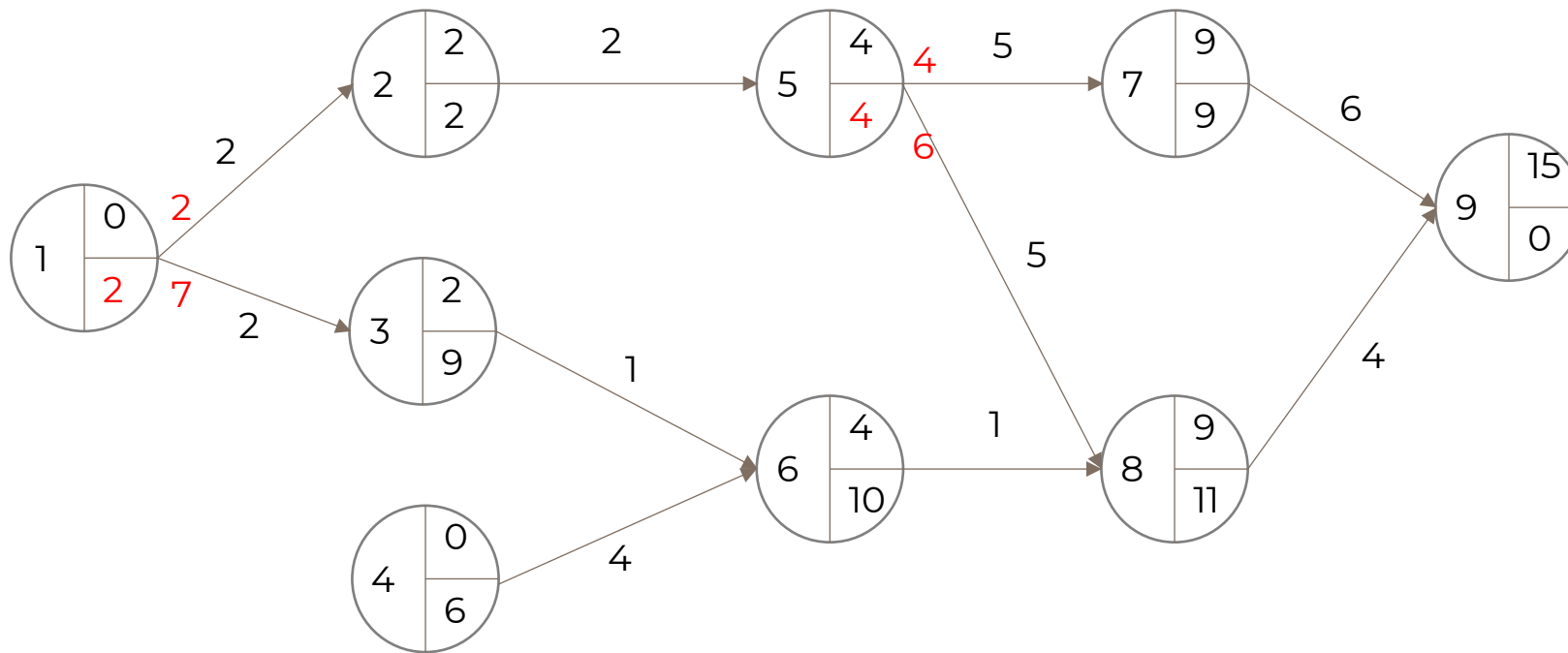
PERT

- 根据各事件持续时间，推算最早开始时间
- 汇总事件取最大值
- 总工期 15
- 最长路径 $1 \rightarrow 2 \rightarrow 5 \rightarrow 7 \rightarrow 9$



PERT

- 根据总工期 15，反推各事件最晚开始时间
- 事件汇总，取最小值



6. 制定进度计划

- 关键路径法



关键路径法

- 可能不止一条
- 时间参数
 - 最早开始时间 ES - Earliest Start Time
 - 最晚开始时间 LS - Latest Start Time
 - 最早完成时间 EF - Earliest Finish Time
 - 最晚完成时间 LF - Latest Finish Time
- 浮动时间 Total Float
 - 总时差
 - $LS - ES = LF - EF$
 - 关键路径上的任务总浮动时间为 0

ES	Duration	EF
	Event	
LS	Total float	LF

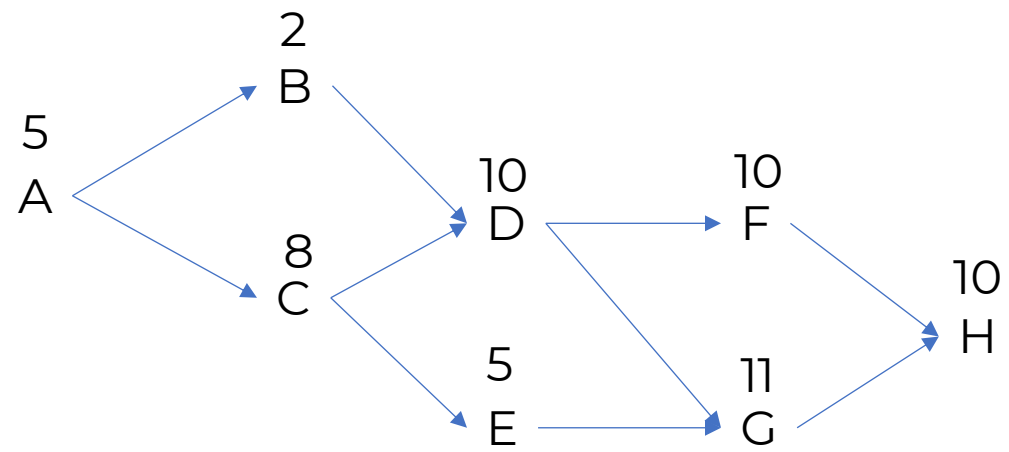


例题
Sample

找出项目的关键路径 Critical Path 和总工期 Project Duration

工作代号	紧前工作	计划工作历时/天
A	——	5
B	A	2
C	A	8
D	B, C	10
E	C	5
F	D	10
G	D, E	11
H	F, G	10





资源优化

- 资源平衡
 - 改变关键路径，通常是延长
- 资源平滑
 - 不会改变关键路径
 - 无法实现所有资源优化



资源压缩

- 赶工 Crashing
 - 加班
- 快速跟进 Fast Tracking
 - 串行 → 并行



7. 进度控制



39、某公司与客户签订了一个系统集成项目合同，对于项目的范围和完成时间做出了明确的规定。在制定进度计划时，项目经理发现按照估算的活动时间和资源编制的进度计划无法满足合同工期，为了达到合同要求，项目经理不宜采用的方法是（ ）。

- A. 赶工 Crashing
- B. 并行施工 Fast Tracking
- C. 增加资源投入
- D. 缩小项目范围

例题
Sample





软件项目质量计划

Section 6 Project Quality



六大质量特性 Quality Characteristics

- 功能性 Functionality
- 可靠性 Reliability
- 易用性 Usability
- 效率 Efficiency
- 可维护性 Maintainability
- 可移植性 Portability



可靠性 Reliability

子特性

成熟性 Maturity

容错性 Fault Tolerance

易恢复性 Recoverability

含义

软件避免由于错误而导致失效的能力。即软件运行稳定，不崩溃

软件在发生故障时，仍能保持功能或自动恢复的能力

软件在发生故障后，能够快速恢复到正常状态的能力



在ISO/IEC 9126软件质量模型中，可靠性质量特性是指在规定的时间内和规定的条件下，软件维持在其性能水平有关的能力，其质量特性不包括（ ）。

- A. 安全性
- B. 成熟性
- C. 容错性
- D. 易恢复性

例题
Sample



质量管理

核心过程

质量规划 Quality Planning

质量保证 Quality Assurance, QA

质量控制 Quality Control, QC

质量改进 Quality Improvement

含义

确定项目质量标准，制定如何满足这些标准的策略

通过审计、流程改进等手段确保质量体系有效运行

监控具体成果是否符合质量标准，识别缺陷并纠正

持续优化流程，提高效率和质量



七大质量管理工具

- 流
- 因果图
- 直方图 Histogram
 - 数据在不同数值区间内出现的频率或次数
- 散点图
- 帕累托定律 Pareto Principle
 - 又称 80/20 法则：大约80%的结果来自20%的原因
- 控制图
- 核查表





软件配置管理计划

Section 7 Project Configuration



小张是软件研发和项目经理，负责的某项目已进入实施阶段，此时用户提出要增加一项新的功能，小张应该（ ）

- A、拒绝该变更
- B、通过变更控制流程进行处理
- C、立即实现该变更
- D、要求客户应先去与公司领导协商

例题
Sample



项目整体变更控制管理的流程是：变更请求 → ()

- A、同意或否决变更 → 变更影响评估 → 执行
- B、执行变更 → 变更影响评估 → 同意或否决变更
- C、变更影响评估 → 同意或否决变更 → 执行
- D、同意或否决变更 → 执行 → 变更影响评估

例题
Sample



- Software Configuration Management
- 是项目管理、系统工程和软件工程中的一个关键过程
- 旨在系统化地识别、控制、记录和审计产品、服务或系统在其生命周期内的功能特性和物理特性，以确保其性能、功能和实体属性与需求、设计和操作信息保持一致
 - 确保产品完整性、可追溯性、一致性和可控性
 - 也称变更管理
- 目标：
 - 保证所有交付物版本可控
 - 支持并行开发协作
 - 快速定位问题根源
 - 满足审计与合规要求

原因

Reason

- 新的业务或市场
- 新的客户需求
- 企业改组或规模调整
- 预算或进度安排受限



六个活动

活动	说明
制定配置管理计划 CMP Configuration Management Plan	为整个项目定义如何实施配置管理的策略、流程、工具与职责
配置标识 CI Configuration Identification	确定哪些工件需要被管理（如代码、文档、测试用例），并赋予唯一标识符（如版本号、基线名）
配置控制 Control	建立变更请求流程（CR）、审批机制、影响分析、实施与验证
状态记账 Status Accounting	记录每个配置项的状态（如草稿、评审中、已批准、已发布）、历史变更记录
配置审计 Audit	功能审计（是否满足需求）+ 物理审计（实际交付物是否与记录一致）



- 基线
- 软件配置项 configuration item



- 涉及的主体
 - 任何干系人 Stakeholder：提出变更申请
 - 项目经理 Project Manager：影响分析；召开变更委员会会议、实施变更
 - 变更控制委员会 CCB, Change Control Board：决策机构；双方决策人；审查、评价、批准
- 基本过程
 - 提交变更请求
 - 进行变更影响评估（由项目经理组织团队分析对范围、进度、成本、质量等的影响）
 - 提交变更控制委员会（CCB, ）审批 → 同意或否决变更
 - 若批准，则执行变更；若拒绝，则通知申请人并归档
 - 更新基线与文档，关闭变更请求

配置管理系统



配置项版本管理

- 主版本号 Major: 重大变更、架构调整、不兼容升级
- 次版本号 Minor: 新增功能、小幅改进
- 修订号 Patch: Bug修复、小调整

状态

草稿 Draft

正式 Formal

修改 Modified

版本号格式

0.Y 或 0.YZ

1.X 或 X.Y

在原基础上递增

说明

初始编写阶段, 尚未评审通过

经过评审、批准后发布

如从 1.0 → 1.1



配置库

库类型	用途	特点
开发库 (Development Library)	存放开发人员正在工作的文件	- 个人或团队工作空间 - 无严格版本控制 - 可自由修改
受控库 (Controlled Library)	存放已提交、需变更控制的配置项	- 经审批后才能修改 - 用于集成、测试 - 实现版本控制
产品库 (Product Library)	存放已发布、正式交付的产品版本	- 最终版本 - 不可随意修改 - 供发布和交付使用



发布管理和交付

活动

打包 Packaging

复制 Replication

存储 Storage

分发 Distribution

部署 Deployment

说明

将多个配置项（如代码、文档、配置文件）组合成一个可发布的单元（如安装包、镜像）

将发布版本复制到目标环境（如测试、生产、客户）

将发布版本存放在产品库或发布服务器中，供后续使用或回滚

向用户或运维团队提供发布包

在目标环境中安装和运行发布包





软件项目团队计划

Section 8 Project Team



团队发展四阶段

- 塔克曼 Bruce Tuckman 于1965年提出

Item	Description
Forming	形成期
Storming	震荡期或磨合期
Norming	规范期
Performing	表现期 / 执行期 / 高绩效期
Adjourning	结束期



人力资源管理

- 编制
- 组建
- 建设
- 管理



责任分配矩阵

人员				
需求定义	张三	李四	王五	赵六
系统设计	▲		○	
系统开发		□		
测试	■		◆	



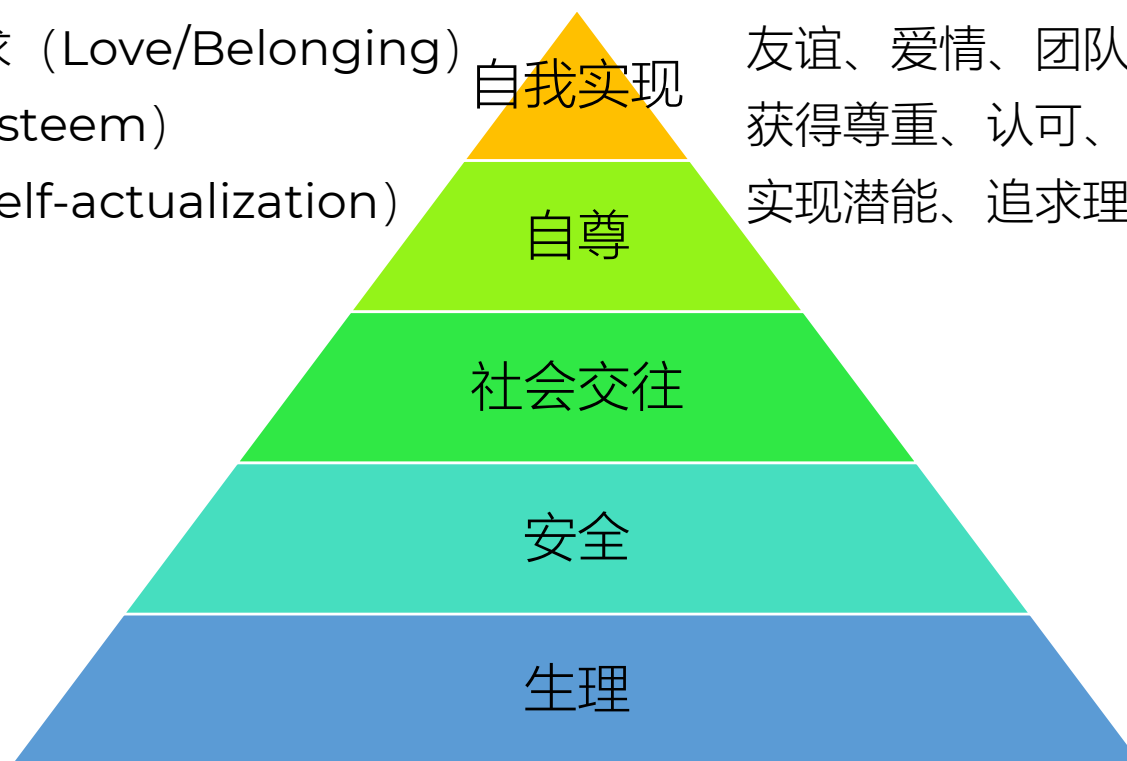
冲突解决

技术	特点	适用场景
回避/撤退 Avoiding/Withdrawal	回避问题，不处理	临时回避，或问题不重要时
安抚/迁就 Accommodating	优先考虑他人需求，牺牲自己	维护关系，暂时让步
妥协 Compromising	双方各让一步，达成折中	时间紧迫，双方都有利害
强制 Forcing	强行推动自己的观点	权力大、必须决策时
合作/解决问题 Collaborating	共同探讨，寻找双赢方案	复杂问题，需要共识



马斯洛需要层次理论

层次	名称	典型表现
1	生理需求 (Physiological)	饮食、睡眠、休息等基本生存需求
2	安全需求 (Safety)	工作稳定、健康保障、职业安全等
3	社会交往需求 (Love/Belonging)	友谊、爱情、团队归属、被接纳
4	自尊需求 (Esteem)	获得尊重、认可、成就感、地位
5	自我实现 (Self-actualization)	实现潜能、追求理想、创造性发挥



赫茨伯格双因素激励理论

- Herzberg's Motivation-Hygiene Theory
- 由美国心理学家弗雷德里克·赫茨伯格（Frederick Herzberg）于20世纪50年代末提出的一种关于工作动机和员工满意度的重要理论
- 激励因素 Motivators - 提升积极性
 - 成就感（Achievement）
 - 认可/赞赏（Recognition）
 - 工作本身（The work itself）
 - 责任感（Responsibility）
 - 晋升机会（Advancement）
 - 成长与发展（Growth）
- 保健因素 Hygiene Factors - 避免不满
 - 公司政策与管理（Company policy）
 - 监督方式（Supervision）
 - 工作条件（Working conditions）
 - 薪资待遇（Salary）
 - 同事关系（Interpersonal relations）
 - 地位（Status）
 - 工作安全感（Job security）



虚拟团队



3、项目团队人员转移首先应满足的条件是（ ）。

A. 项目人力资源管理计划中所描述的人员转移条件已触发

 B.  项目团队成员所承担的任务已完成，提交了经过确认的可交付物并已完成工作交接

C. 项目经理与项目团队成员确认该成员的工作衔接已经告一段落或者已经完成

D. 项目经理签发项目团队成员转移确认文件

沟通管理 Communication Mgt

- 规划管理过程
- 管理沟通
- 控制管理过程
 - 有效沟通：效果和效率

类型	定义	特点
A. 正式沟通	通过组织结构、书面文件、会议等正规渠道进行	规范、有记录、可控性强
B. 非正式沟通	口头交流、闲聊、社交互动等非官方形式	灵活、快速、但可能失真
C. 垂直沟通	上下级之间的沟通（如项目经理与下属）	权责明确，流程清晰
D. 水平沟通	同级之间或跨部门之间的沟通	协调难度大，易受多方影响



干系人管理 Stakeholders Mgt

- 典型流程：
 - 识别干系人 Identify Stakeholders - 找出谁是干系人，并收集基本信息（如姓名、角色、部门）
 - 评估干系人 Analyze Stakeholders - 分析他们的诉求、期望、影响力、权力等
 - 对干系人分类 Categorize Stakeholders - 使用方格模型（如权利/利益方格）进行归类
 - 制定干系人管理计划 Develop Stakeholder Management Plan - 根据分类结果，制定沟通策略、管理方式等

输入

说明

项目章程

包含高层级干系人信息，如发起人、客户等

采购文件

如合同、招标书，涉及供应商、承包商等外部干系人

干系人记录模板

组织内部使用的标准化表格，用于记录干系人信息

组织过程资产

包括以往项目的干系人清单、模板、经验教训等



凸显模型 Salience Model

维度	含义
权力 Power	是否有能力影响项目决策或资源分配？
合法性 Legitimacy	其参与是否符合规则、道德或社会期望？
紧迫性 Urgency	是否需要立即响应？ 是否有时间压力？



干系人 Stakeholders

编号	英文名称	中文名称	特征说明
1	Dormant Stakeholders	潜伏型	有权力但不主动，合法且非紧急。可暂时忽略，但需监控
2	Discretionary Stakeholders	随意型	无权力，但合法且紧急。例如：临时求助的同事
3	Demanding Stakeholders	娇情型	有权、合法但不紧急。喜欢提要求，但不会立刻施压
4	Dominant Stakeholders	权贵型	有权、合法、但不紧急。典型如高层领导、董事会成员
5	Dangerous Stakeholders	危险型	有权、不合法、但紧急。可能违法或破坏项目，需警惕
6	Dependent Stakeholders	从众型	无权、不合法、也不紧急。依赖他人支持，影响力小
7	Definitive Stakeholders	统治型	三者俱全：有权、合法、紧急。最核心的干系人，必须重点管理



干系人 Stakeholders

- 利益方格 Power/Interest C
- Manage closely



权力 Power

高

高

低

低

利益 Interest

高

低

高

低

管理策略

重点管理 Manage closely

监督 Keep satisfied

随时告知 Keep informed

令其满意 Minimal effort





1. 类型
2. 3要素
3. 4过程

软件项目风险计划

Section 9 Project Risk



- 风险是对潜在的、未来可能发生损害的一种度量
- 软件风险对软件开发过程及软件产品本身可能造成的伤害或损失。

风险曝光度 Risk Exposure

- 风险概率*可能的损失
- 管理风险可能会带来新的风险
- 风险是客观存在的，不可能完全避免
- 风险管理应贯穿整个项目管理过程



在风险管理中，通常需要进行风险监测，其目的不包括（ ）。

- A. 消除风险
- B. 评估所预测的风险是否发生
- C. 保证正确实施了风险缓解步骤
- D. 收集用于后续进行风险分析的信息

例题
Sample

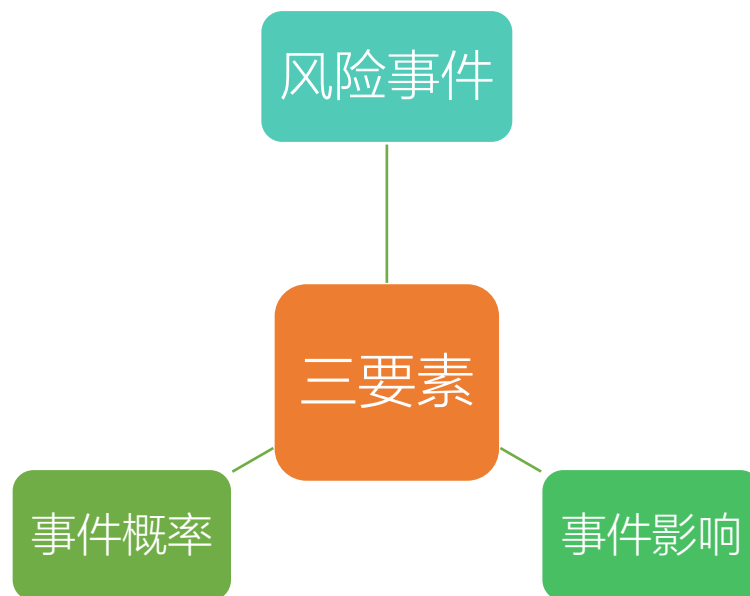


预测角度

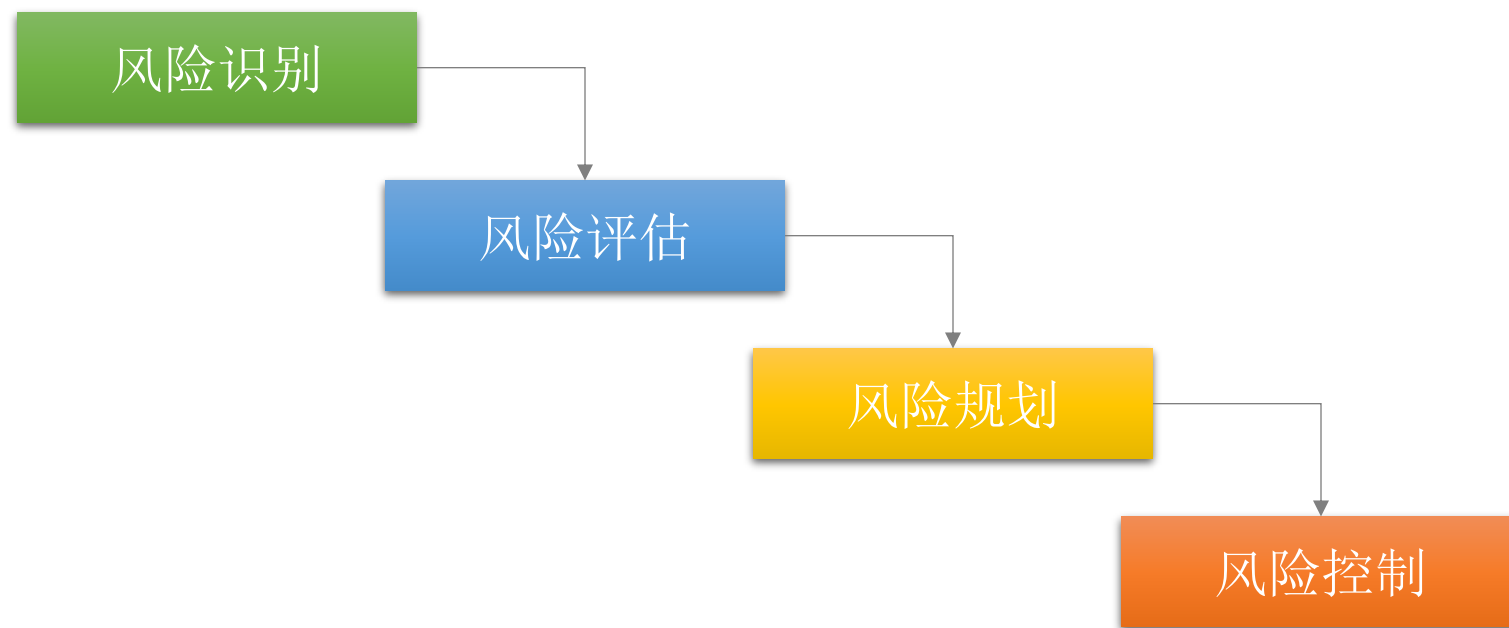
已知风险 Known known
可预测风险 Known unknown
不可预测风险 unknown unknown

范围角度

商业风险、管理风险、人员风险、技术风险、开发环境风险、客户风险、过程风险、产品规模风险等。



风险4个过程



SWOT

字母	含义	内容
S	Strengths (优势)	组织内部的积极因素 (如技术强、团队经验丰富)
W	Weaknesses (劣势)	组织内部的消极因素 (如资源不足、流程混乱)
O	Opportunities (机会)	外部环境中的有利条件 (如政策支持、市场需求增长)
T	Threats (威胁)	外部环境中的不利因素 (如竞争加剧、法规变化)



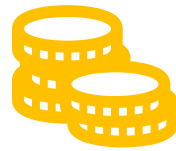


1. 类型
2. 签订
3. 管理

软件项目合同计划

Section 10 Project Contract





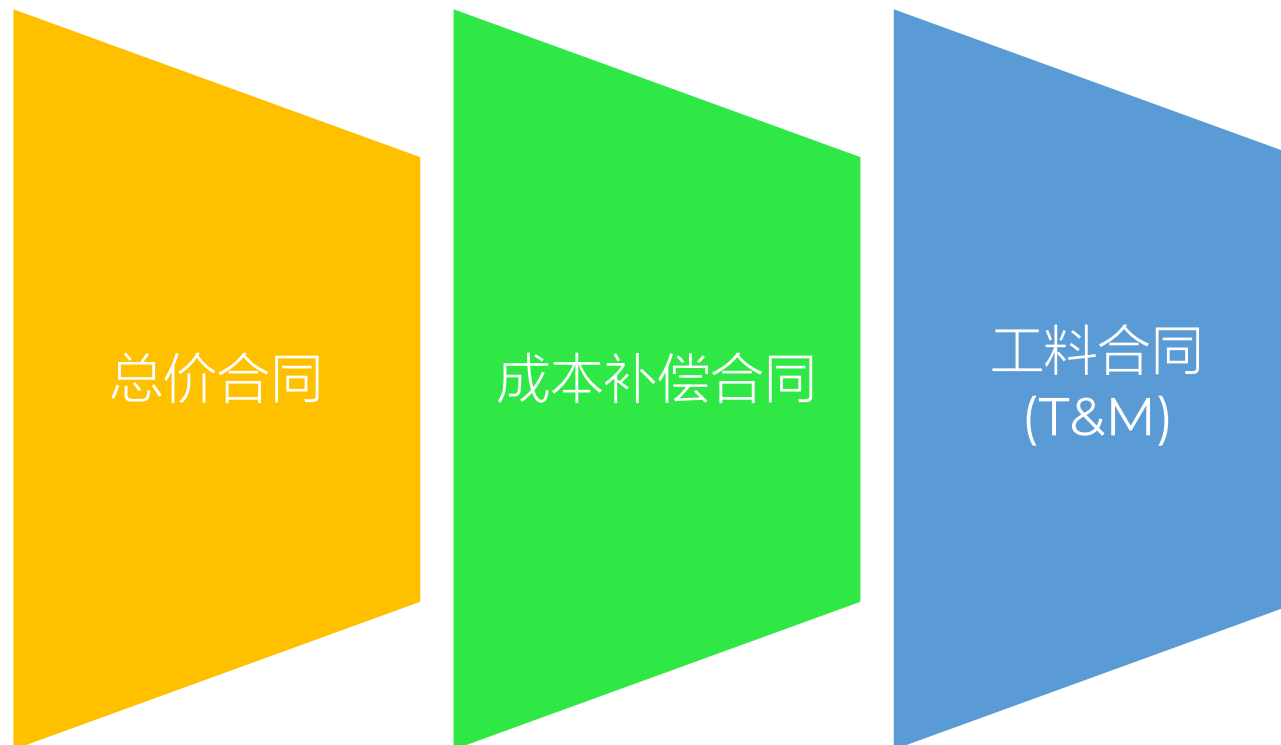
- 合同是具有法律效力的协议
- 双方自愿达成的协议
- 签订者具有相应的法律能力
- 有充分的签约理由
- 具有合法的目的



- 按范围方式
 - 总承包合同
 - 单项工程承包合同
 - 分包合同



- 按支付方式



- Fixed Price
- 总价合同为既定产品、服务或成果的采购设定一个总价
- 预算固定、成本可控
- 适用：需求明确，且不会出现重大范围变更。



- Cost Reimbursable
- 向卖方支付为完成工作而发生的全部合法实际成本，外加一笔费用作为卖方的利润。
- 组成：实际成本+酬金/利润
- 适用：工作范围预计会在合同执行期间发生重大变更

- Time & Material, T&M
- 兼具：成本补偿合同和总价合同的特点
- 组成：实际成本+管理
- 必须为每一个单位的工作量付出一定的报酬
- 适用范围：宽

合同类型	特点	适用场景
总价合同	单价或总价固定，风险由卖方承担	范围明确、需求清晰
成本补偿合同	支付实际成本+酬金，买方承担大部分风险	范围不确定、需灵活调整
工料合同	按工时和材料计费，单价固定，总量不固定	范围不清但可定义工作单元
采购单合同 Purchase Order	简单采购订单，用于一次性商品/服务	小额、标准品、非复杂项目



- 明确如何进行委托、委托什么项目、何时进行、费用如何等
- 选择需要的合同类型，采用的招标方式、合同形式等
- 输出
 - 招标书或者类似招标书的形式体现的



- 多层协议结构
- 价值交付（例如迭代付费）
- 总价增量（例如基于Story付费）
- 灵活总价方案
- 动态范围方案



合同索赔

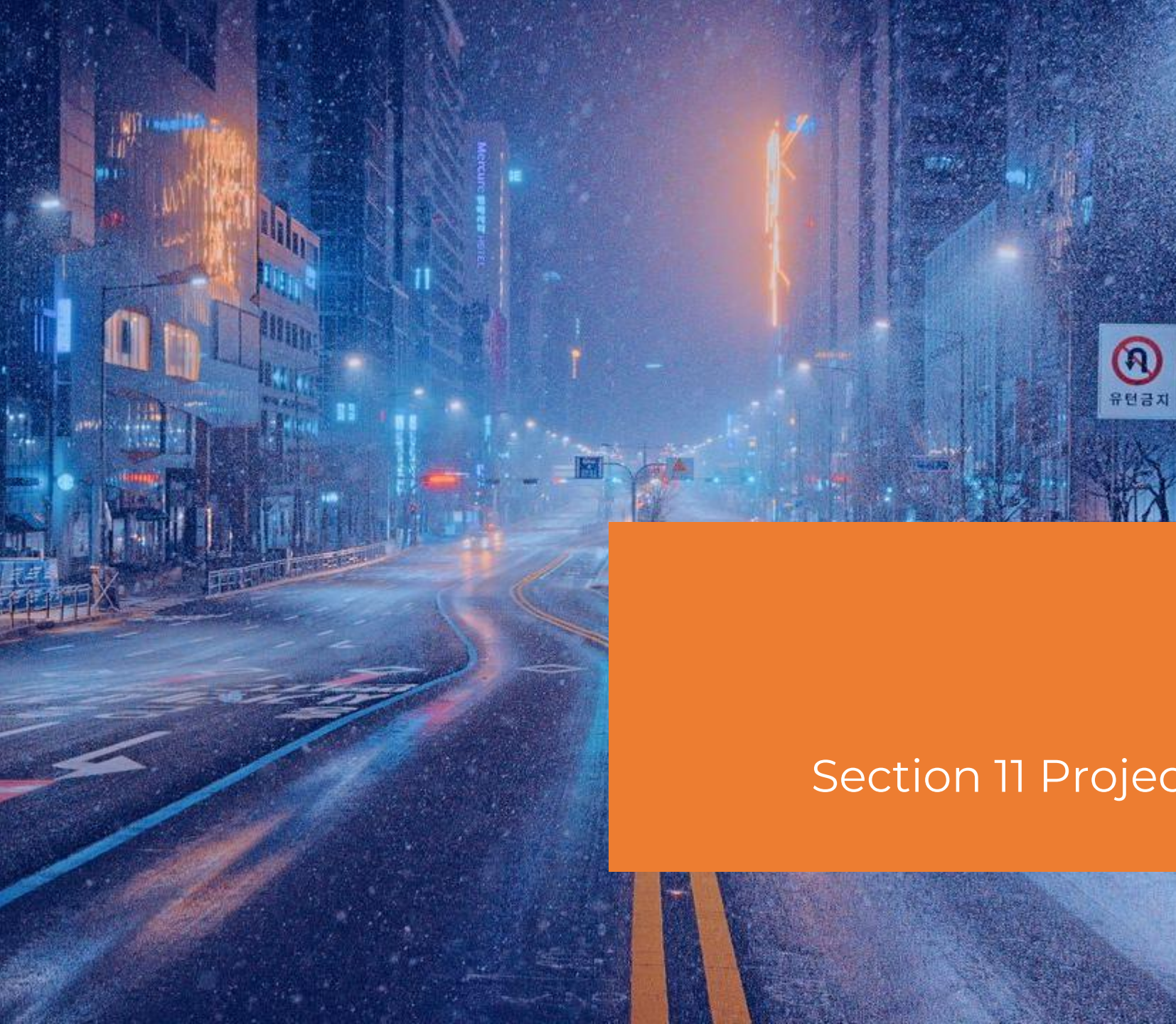
- 经济补偿，不是处罚
- 分类：
 - 工期索赔
 - 费用索赔
- 期限：每个阶段均为28天



合同变更

- 合同变更是一个管理闭环过程，通常遵循以下顺序：
 - 变更提出 Change Request Submitted: 一方（如业主、承包商）提出变更需求
 - 变更请求审查 Review of Change Request: 对变更的影响进行评估（成本、工期、质量等）
 - 变更批准 Approval of Change: 经审批后决定是否接受变更
 - 变更实施 Implementation of Change: 执行变更内容，并记录结果





项目执行控制

Section 11 Project Execution & Control

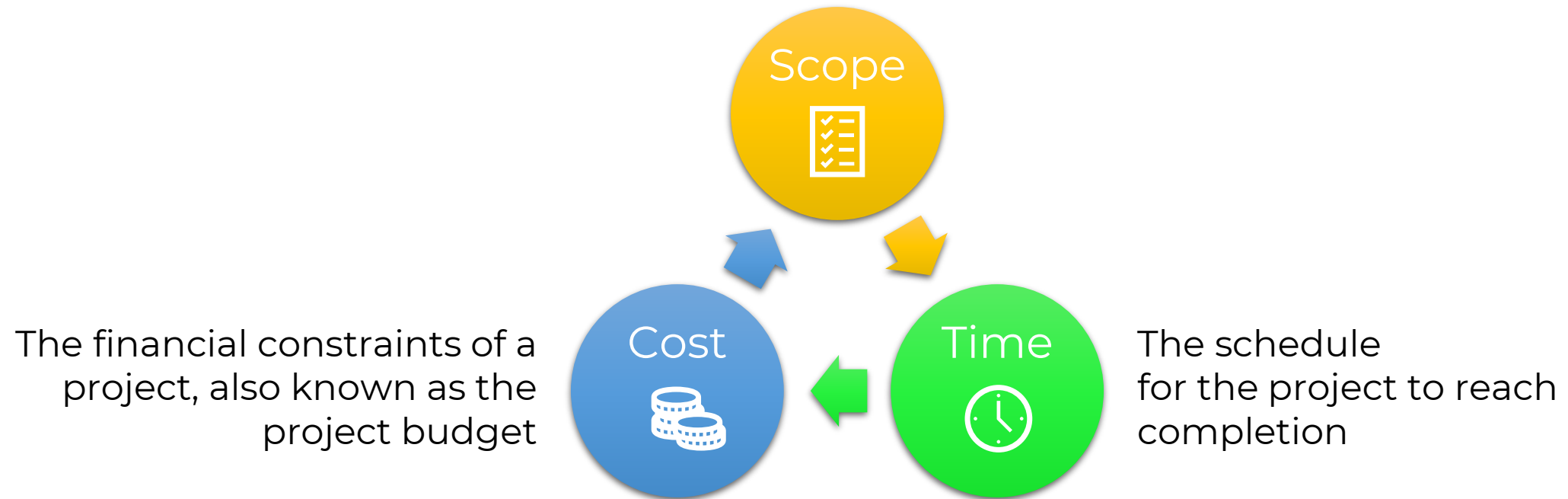


-
- 马云和比尔盖茨都表达过类似的观点：“一流的战略、三流的执行”不如“三流战略、一流的执行”。一流的战略和一流的执行，似乎就像鱼与熊掌一样不可兼得。



Triple Constraint Diagram

The tasks
required to fulfill the project's goals



Construction project Triple Constraint Example

- 假设正在建设一个小型零售店，大约2,000平方英尺。项目时间为6个月，预算为30万美元
- 如果客户在最后一刻提出增加500平方英尺的要求：
 - 扩大项目范围
 - 要么时间表必须推迟，要么预算增加



IT project Triple Constraint Example

- 如果一个销售团队需要建立一个新的客户关系管理系统CRM，他们可能有9个月的时间框架和25万美元的预算。在范围方面，CRM需要包括潜在客户管理、销售报告和联系跟踪
- 如果销售团队要求系统在6个月内准备就绪：
 - 减少范围
 - 增加预算



Manufacturing Triple Constraint Example

- 假设一家制造公司正在生产5,000台新型智能家居设备。它需要包含应用程序集成和语音控制，每台设备的制造成本不能超过50美元，并且所有设备必须在3个月内生产完成
- 如果预算中突然出现成本变化，并且每台设备的制造成本不能超过40美元：
 - 要么需要放慢速度，这将延长预算
 - 要么范围将需要缩减





1. 项目终止
2. 项目结束

项目结束

Section 12 Project Closure



项目终止条件

- 项目计划中确定的可交付成果已经出现
 - 项目的目标已经成功实现
- 由于相关原因,项目无法继续进行
 - 项目已经不具备实用价值
 - 项目无竞争力, 难以生存
 -

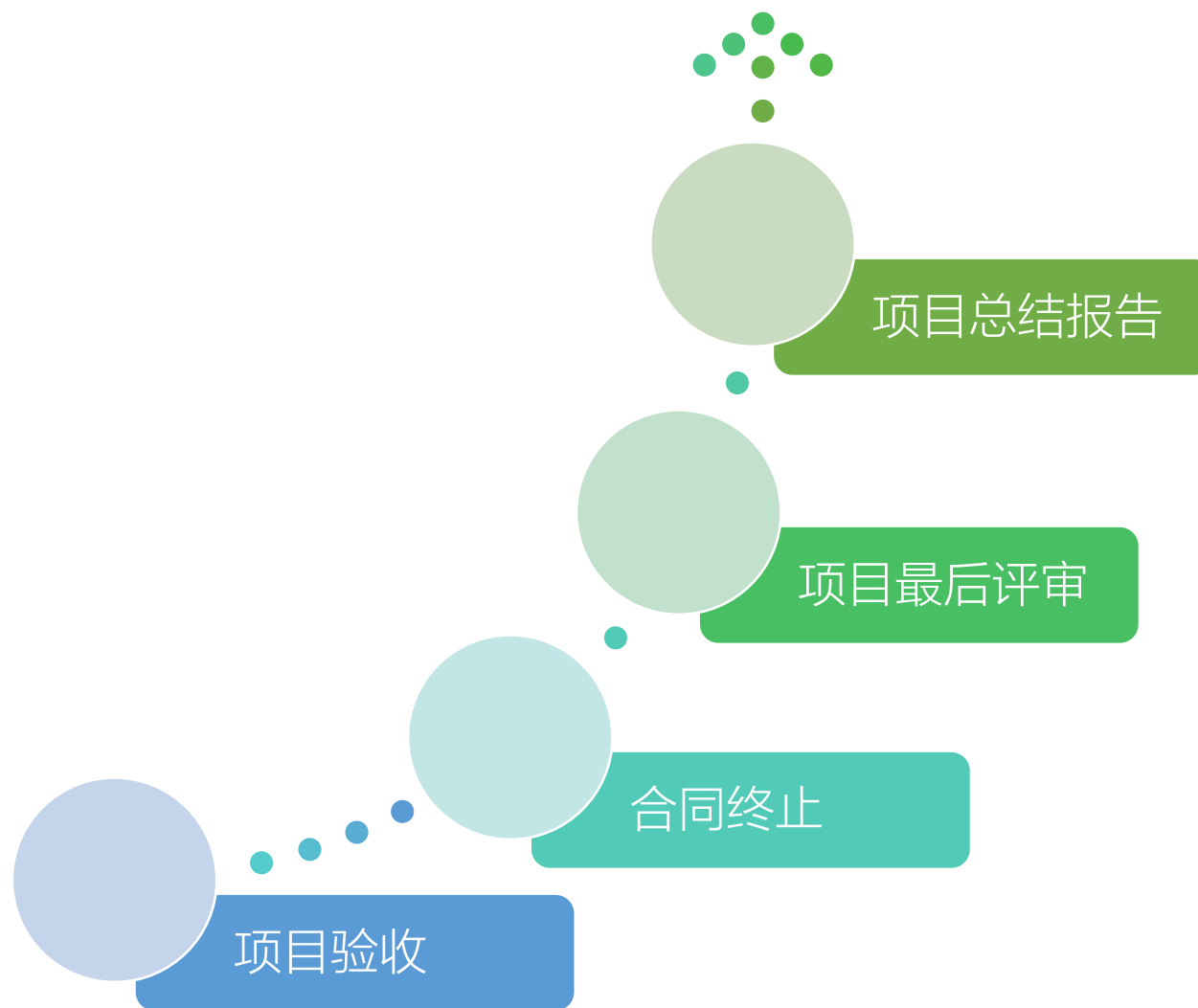


项目终止过程

- 达到项目的完工或退出标准
- 完成项目合同协议
- 完成项目信息报告



项目结束



项目结束

- 合同结束
- 行政结束



项目验收

- 验收测试
- 系统试运行
- 文档验收
- 项目终验

(3) 系统文档验收

对于系统集成项目，所涉及的文物应该包括如下部分：

- 1) 系统集成项目介绍
- 2) 系统集成项目最终报告
- 3) 信息系统说明手册
- 4) 信息系统维护手册
- 5) 软硬件产品说明书，质量保证书等

全套



(4) 项目终验



项目总结

- 管理收尾/行政收尾



项目后评价

- 目标评价
- 过程评价
- 效益评价
- 可持续评价

信息系统后评价通过对已经建成的信息系统进行全面综合地调研，分析，总结，对信息系统目标是否实现，信息系统的前期论证过程，开发建设过程以及运营外维护过程是否符合要求，信息系统是否实现了预期的经济效益，管理效益以及的社会效益，信息系统后评价的主要内容一般包括信息系统的目标评价，信息系统过程评价，信息系统效益评价和信息系统可持续性评价四个方面的工作内容。



Tips

- 聚焦价值
- 正能量
- 主动
- 客观

